

Fenix FEM

Manual de Referencia

Especificaciones físicas, formulaciones numéricas y contratos
de los componentes del programa

Generado automáticamente desde docs/specs/

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

13 de mayo de 2026

Índice general

Sobre este Manual	VII
1. Elementos 1D — Armaduras	1
1.1. Truss2D	1
1.2. Especificación física	1
1.2.1. 0. Descripción general	1
1.2.2. 1. Cinemática de desplazamientos	1
1.2.3. 2. Cinemática de deformaciones	1
1.2.4. 3. Ecuación constitutiva	1
1.2.5. 4. Equilibrio — forma fuerte	1
1.2.6. 5. Equilibrio — forma débil	2
1.3. Formulación numérica (FEM)	2
1.3.1. 6. Discretización	2
1.3.2. 7. Funciones de forma	2
1.3.3. 8. Matriz deformación-desplazamiento	2
1.3.4. 9. Rigidez elemental	2
1.3.5. 10. Fuerzas internas	2
1.3.6. 11. Cuadratura	2
1.4. Contrato de implementación	2
1.5. Implementación	3
1.6. Diálogo	4
1.7. Truss2DCorot	5
1.8. Especificación física	5
1.8.1. 0. Descripción general	5
1.8.2. 1. Cinemática	5
1.8.3. 2. Medida de deformación	5
1.8.4. 3. Ecuación constitutiva	5
1.8.5. 4. Equilibrio — forma débil incremental	5
1.9. Formulación numérica (FEM)	6
1.9.1. 6. Discretización	6
1.9.2. 7. Vector dirección actual	6
1.9.3. 8. Matriz B corotacional	6
1.9.4. 9. Rigidez tangente	6
1.9.5. 10. Fuerzas internas	6
1.9.6. 11. Cuadratura	6
1.10. Contrato de implementación	6
1.11. Implementación	7

1.12. Diálogo	8
1.13. Truss3D	9
1.14. Especificación física	9
1.14.1. 0. Descripción general	9
1.14.2. 1. Cinemática de desplazamientos	9
1.14.3. 2. Cinemática de deformaciones	9
1.14.4. 3. Ecuación constitutiva	9
1.14.5. 4. Equilibrio — forma fuerte	9
1.14.6. 5. Equilibrio — forma débil	9
1.15. Formulación numérica (FEM)	10
1.15.1. 6. Discretización	10
1.15.2. 7. Funciones de forma	10
1.15.3. 8. Matriz deformación-desplazamiento	10
1.15.4. 9. Rigidez elemental	10
1.15.5. 10. Fuerzas internas	10
1.15.6. 11. Cuadratura	10
1.16. Contrato de implementación	10
1.17. Implementación	11
1.18. Diálogo	12
1.19. Truss3DCorot	13
1.20. Especificación física	13
1.20.1. 0. Descripción general	13
1.20.2. 1. Cinemática	13
1.20.3. 2. Medida de deformación	13
1.20.4. 3. Ecuación constitutiva	13
1.20.5. 4. Equilibrio — forma débil incremental	13
1.21. Formulación numérica (FEM)	13
1.21.1. 6. Discretización	13
1.21.2. 7. Vector dirección actual	14
1.21.3. 8. Matriz B corotacional	14
1.21.4. 9. Rigidez tangente	14
1.21.5. 10. Fuerzas internas	14
1.21.6. 11. Cuadratura	14
1.22. Contrato de implementación	14
1.23. Implementación	15
1.24. Diálogo	16
2. Elementos 1D — Cables	17
2.1. Cable2DCorot	17
2.2. Especificación física	17
2.2.1. 0. Descripción general	17
2.2.2. 1. Cinemática	17
2.2.3. 2. Medida de deformación	17
2.2.4. 3. Ecuación constitutiva	18
2.2.5. 4. Equilibrio — forma débil incremental	18
2.3. Formulación numérica (FEM)	18
2.3.1. 6. Discretización	18
2.3.2. 7. Vectores de dirección (configuración corriente)	18

2.3.3.	8. Matriz B corotacional	18
2.3.4.	9. Rigidez tangente	18
2.3.5.	10. Fuerzas internas	19
2.3.6.	11. Cuadratura	19
2.4.	Contrato de implementación	19
2.5.	Implementación	20
2.6.	Diálogo	21
2.7.	Cable3DCorot	22
2.8.	Especificación física	22
2.8.1.	0. Descripción general	22
2.8.2.	1. Cinemática	22
2.8.3.	2. Medida de deformación	22
2.8.4.	3. Ecuación constitutiva	22
2.8.5.	4. Equilibrio — forma débil incremental	22
2.9.	Formulación numérica (FEM)	23
2.9.1.	6. Discretización	23
2.9.2.	7. Vector dirección actual	23
2.9.3.	8. Matriz B corotacional	23
2.9.4.	9. Rigidez tangente	23
2.9.5.	10. Fuerzas internas	23
2.9.6.	11. Cuadratura	23
2.10.	Contrato de implementación	23
2.11.	Implementación	25
2.12.	Diálogo	26
3.	Elementos 1D — Marcos / Vigas	27
3.1.	Frame2DEuler	27
3.2.	Especificación física	27
3.2.1.	0. Descripción general	27
3.2.2.	1. Hipótesis cinemática de Euler-Bernoulli	27
3.2.3.	2. Campos de desplazamiento	27
3.2.4.	3. Medidas de deformación	28
3.2.5.	4. Ecuaciones constitutivas	28
3.2.6.	5. Equilibrio — forma débil	28
3.3.	Formulación numérica (FEM)	28
3.3.1.	6. Discretización	28
3.3.2.	7. Sistema local y transformación	28
3.3.3.	8. Funciones de forma	29
3.3.4.	9. Rigidez local	29
3.3.5.	10. Fuerzas internas	29
3.3.6.	11. Ensamblaje global	29
3.3.7.	12. Cuadratura	29
3.4.	Contrato de implementación	30
3.5.	Implementación	31
3.6.	Diálogo	31
3.7.	Frame2DTimoshenko	33
3.8.	Especificación física	33
3.8.1.	0. Descripción general	33

3.8.2.	1. Hipótesis cinemática de Timoshenko	33
3.8.3.	2. Campos de desplazamiento	33
3.8.4.	3. Medidas de deformación	33
3.8.5.	4. Ecuaciones constitutivas	34
3.8.6.	5. Factor de Phi (shear locking)	34
3.8.7.	6. Equilibrio — forma débil	34
3.9.	Formulación numérica (FEM)	34
3.9.1.	7. Discretización	34
3.9.2.	8. Sistema local y transformación	34
3.9.3.	9. Rigidez local con corrección por shear locking	35
3.9.4.	10. Fuerzas internas	35
3.9.5.	11. Ensamblaje global	35
3.9.6.	12. Cuadratura	35
3.10.	Contrato de implementación	35
3.11.	Implementación	36
3.12.	Diálogo	37
3.13.	Frame2DEulerCorot	38
3.14.	Especificación física	38
3.14.1.	0. Descripción general	38
3.14.2.	1. Cinemática corotacional	38
3.14.3.	2. Medidas de deformación	38
3.14.4.	3. Ecuaciones constitutivas	38
3.14.5.	4. Equilibrio — forma débil	39
3.15.	Formulación numérica (FEM)	39
3.15.1.	5. DOFs locales deformacionales	39
3.15.2.	6. Rigidez local	39
3.15.3.	7. Matriz de transformación $\mathbf{B}$ (3×6)	39
3.15.4.	8. Rigidez tangente	39
3.15.5.	9. Cuadratura	40
3.16.	Contrato de implementación	40
3.17.	Implementación	41
3.18.	Diálogo	42
3.19.	Frame3D	43
3.20.	Especificación física	43
3.20.1.	0. Descripción general	43
3.20.2.	1. Hipótesis	43
3.20.3.	2. Sistema de ejes locales	43
3.20.4.	3. Medidas de deformación / acciones internas	43
3.20.5.	4. Equilibrio — forma débil	44
3.21.	Formulación numérica (FEM)	44
3.21.1.	5. Discretización	44
3.21.2.	6. Construcción de ejes locales	44
3.21.3.	7. Matriz de transformación global $\rightarrow$ local	44
3.21.4.	8. Matriz de rigidez local	45
3.21.5.	9. Fuerzas internas	45
3.21.6.	10. Ensamblaje global	45
3.21.7.	11. Cuadratura	45
3.22.	Contrato de implementación	45

3.23. Implementación	47
3.24. Diálogo	48
4. Elementos 2D — Sólidos	49
4.1. Quad4	49
4.2. Especificación física	49
4.2.1. 0. Descripción general	49
4.2.2. 1. Cinemática de desplazamientos	49
4.2.3. 2. Cinemática de deformaciones	49
4.2.4. 3. Ecuación constitutiva	49
4.2.5. 4. Equilibrio — forma fuerte	50
4.2.6. 5. Equilibrio — forma débil	50
4.3. Formulación numérica (FEM)	50
4.3.1. 6. Discretización	50
4.3.2. 7. Funciones de forma	50
4.3.3. 8. Matriz deformación-desplazamiento	50
4.3.4. 9. Rigidez elemental	50
4.3.5. 10. Fuerzas internas	51
4.3.6. 11. Cuadratura	51
4.3.7. 12. Cargas distribuidas consistentes	51
4.3.8. 13. Salida por punto de Gauss	51
4.4. Contrato de implementación	52
4.5. Implementación	53
4.6. Diálogo	53
4.7. Tri3	55
4.8. Especificación física	55
4.8.1. 0. Descripción general	55
4.8.2. 1. Cinemática de desplazamientos	55
4.8.3. 2. Cinemática de deformaciones	55
4.8.4. 3. Ecuación constitutiva	55
4.8.5. 4. Equilibrio — forma fuerte	55
4.8.6. 5. Equilibrio — forma débil	55
4.9. Formulación numérica (FEM)	55
4.9.1. 6. Discretización	55
4.9.2. 7. Funciones de forma	56
4.9.3. 8. Matriz deformación-desplazamiento	56
4.9.4. 9. Rigidez elemental	56
4.9.5. 10. Fuerzas internas	56
4.9.6. 11. Cuadratura	56
4.9.7. 12. Cargas distribuidas consistentes	56
4.9.8. 13. Salida por punto de Gauss	57
4.10. Limitación crítica — <i>shear locking</i>	57
4.11. Contrato de implementación	57
4.12. Implementación	58
4.13. Diálogo	58

5. Modelos Constitutivos (Materiales)	60
5.1. CableMaterial1D	60
5.2. Especificación física	60
5.2.1. 0. Descripción general	60
5.2.2. 1. Ecuación constitutiva	60
5.2.3. 2. Módulo tangente	60
5.2.4. 3. Variables internas	60
5.2.5. 4. Interpretación física	61
5.3. Contrato de implementación	61
5.4. Implementación	62
5.5. Diálogo	63
6. Modelos constitutivos — catálogo	64
6.1. Elastic1D — elástico lineal 1D	64
6.2. Elastic2D — elástico lineal 2D	64
6.3. IsotropicDamage1D — daño isótropo 1D con softening exponencial	65
6.4. IsotropicDamage2D — daño isótropo 2D con softening exponencial	65
6.5. Elastoplastic1D — plasticidad J2 1D con endurecimiento isótropo lineal	66
6.6. CableMaterial1D — cable axial 1D con elasticidad unilateral	66
6.7. VonMises2D — plasticidad J2 2D con endurecimiento isótropo lineal	67
6.8. Cómo añadir un material nuevo	68
7. Anexos técnicos	69
7.1. Dos capas que se llaman ambas «solver»	69
7.2. Backends disponibles y criterio de selección	69
7.3. Fallback automático SPD $\rightarrow$ LU	70
7.4. Factorización reusable y Newton modificado	70
7.5. Override desde YAML — herramienta de diagnóstico	71
7.6. Activación de Cholesky (opcional)	71

Sobre este Manual

Este manual contiene la **referencia formal** de los componentes de Fenix FEM: cada elemento finito y cada modelo constitutivo se documenta con su especificación física (ecuaciones), su formulación numérica (matrices **B**, rigidez tangente, integración) y su contrato YAML.

El contenido se genera automáticamente desde los archivos `docs/specs/*.md` del repositorio, que constituyen la *fuentes única de verdad* de cada componente. No editar este PDF manualmente; cualquier corrección debe hacerse sobre la spec correspondiente y regenerar el manual mediante:

```
python manuals/build_reference_manual.py
```

Para una guía orientada al uso del programa (sintaxis YAML, ejemplos, post-procesamiento), consulte el **Manual de Usuario** en `manuals/User_manual.pdf`.

Capítulo 1

Elementos 1D — Armaduras

1.1 Truss2D

*Orden de trabajo. El usuario escribe la **especificación física**, la **formulación numérica** y el **contrato**. La IA rellena **implementación** y responde en **diálogo** durante el trabajo.*

1.2 Especificación física

1.2.1 0. Descripción general

Elemento sólido 1D de primer orden, inmerso en el plano. Dos nodos articulados; transmite únicamente esfuerzo axial. Aproximación C^0 del desplazamiento.

1.2.2 1. Cinemática de desplazamientos

Interpolación lineal del desplazamiento axial a lo largo del eje $s \in [0, L]$:

$$u(s) = a_0 + a_1 s$$

1.2.3 2. Cinemática de deformaciones

Única componente (axial, en el sistema local de la barra):

$$\varepsilon_{ss}(s) = \frac{du}{ds}$$

1.2.4 3. Ecuación constitutiva

$$\sigma_{ss} = E \varepsilon_{ss}$$

1.2.5 4. Equilibrio — forma fuerte

$$-\frac{d}{ds} \left(EA \frac{du}{ds} \right) = b(s), \quad s \in (0, L),$$

con condiciones de contorno Dirichlet o Neumann ($\sigma A = \bar{F}$) en los extremos.

1.2.6 5. Equilibrio — forma débil

$$\int_0^L \frac{d\delta u}{ds} EA \frac{du}{ds} ds = \int_0^L \delta u b ds + [\delta u \bar{F}]_{\partial\Omega}, \quad \forall \delta u \in V_0.$$

1.3 Formulación numérica (FEM)

1.3.1 6. Discretización

Dos nodos en $s = 0$ y $s = L$. Cosenos directores del eje en globales:

$$c = \frac{x_2 - x_1}{L}, \quad s_\theta = \frac{y_2 - y_1}{L}.$$

1.3.2 7. Funciones de forma

$$N_1(s) = 1 - \frac{s}{L}, \quad N_2(s) = \frac{s}{L}.$$

1.3.3 8. Matriz deformación-desplazamiento

Local (1 DOF axial por nodo): $\mathbf{B}_{\text{loc}} = \frac{1}{L}[-1, 1]$. Global ($\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top$):

$$\mathbf{B} = \frac{1}{L}[-c, -s_\theta, c, s_\theta], \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.3.4 9. Rigidez elemental

Integración analítica exacta (EA constante):

$$\mathbf{K}_e = \int_0^L \mathbf{B}^\top EA \mathbf{B} ds = \frac{EA}{L} \mathbf{d}^\top \mathbf{d}, \quad \mathbf{d} = [-c, -s_\theta, c, s_\theta].$$

1.3.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sigma A \mathbf{d}.$$

1.3.6 11. Cuadratura

No aplica (forma cerrada).

1.4 Contrato de implementación

```
name: Truss2D
kind: element
status: validated          # draft → implemented → validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
```

```

n_integration_points: 1

parameters:
- { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
signature: "compute_state(, state) -> (, E_tangent, state')"
strain_kind: axial escalar

conventions:
sign: " > 0 > 0 (elongación)"
voigt: "N/A (escalar)"
node_orientation: "libre; K_e y F_int invariantes bajo permutación"

validity:
- "|| 1e-2"
- pequeños desplazamientos y rotaciones
- cargas exclusivamente axiales

out_of_scope:
- flexión, cortante, momentos en extremos
- grandes desplazamientos
- pandeo

acceptance:
- name: barra axial bajo carga puntual
setup: "empotrada en s=0, carga F axial en s=L"
expect: "u(L) = F·L / (E·A)"
tol_rel: 1.0e-10

references:
- "Bathe K.-J., Finite Element Procedures, §4.2.1"

```

1.5 Implementación

- **Archivo:** fenix/elements/truss.py
- **Clase:** Truss2D (registrada vía `@ElementRegistry.register`)
- **Tests:**
 - tests/test_truss.py · `TestTruss2D` — verifica $L0$, cosenos directores, entradas de $\mathbf{K}_e$ (incluyendo simetría), y evaluación de $\mathbf{F}_{\text{int}}$ sobre desplazamientos conocidos. Los valores numéricos esperados coinciden con $\mathbf{K}_e = (EA/L)\mathbf{dd}^\top$ y $\mathbf{F}_{\text{int}} = N\mathbf{d}$ para geometría de prueba $L = 5$, $c = 0,6$, $s = 0,8$.
 - tests/test_integration.py · `TestSolversIntegration` — resuelve end-to-end el caso de aceptación (barra empotrada en un extremo, carga axial en el otro) con `NonlinearSolver` y `ArcLengthSolver`; en régimen elástico precedente a la fluencia reproduce $u(L) = FL/(EA)$.
- **Notas de traducción:**
 - $L0$, c , s se calculan una vez en `__init__` sobre la configuración inicial y no se actualizan — coherente con el régimen infinitesimal declarado.
 - El contrato con el material es el estándar del proyecto: `material.compute_state( $\epsilon$ , state) → ( $\sigma$ ,  $E_t$ , state')`.

- Para grandes desplazamientos/rotaciones usar `Truss2DCorot`, que hereda de esta clase.

1.6 Diálogo

- **2026-04-21** · Cierre de ciclo retroactivo. La clase `Truss2D` preexistía al flujo spec-first; la spec se creó documentando la formulación ya implementada. Los tests unitarios y de integración satisfacen el único criterio de **acceptance** declarado (analíticamente, vía los valores de $\mathbf{K}_e$ y end-to-end vía el solver). Promovido **status: validated**.

1.7 Truss2DCorot

*Orden de trabajo. Usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

1.8 Especificación física

1.8.1 0. Descripción general

Barra articulada 2D equivalente a **Truss2D** pero en régimen de **grandes desplazamientos y rotaciones** con **pequeña deformación axial**. La cinemática se actualiza en configuración corriente (corotacional / Updated Lagrangian): el eje de la barra gira con el movimiento y la deformación se mide sobre la longitud actual. Rigidez tangente = rigidez material + rigidez geométrica debida al esfuerzo axial.

1.8.2 1. Cinemática

Configuraciones:

- Referencia: nodos $\mathbf{X}_1, \mathbf{X}_2$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Cosenos directores en configuración corriente:

$$c_\theta = \frac{x_2 - x_1}{l}, \quad s_\theta = \frac{y_2 - y_1}{l}.$$

1.8.3 2. Medida de deformación

Ingenieril corotacional (válida para pequeña deformación aunque la rotación sea grande):

$$\varepsilon = \frac{l - L_0}{L_0}.$$

1.8.4 3. Ecuación constitutiva

Material 1D: $\sigma = E \varepsilon$ (elástico lineal); el contrato admite cualquier material con **STRAIN_DIM** = 1.

1.8.5 4. Equilibrio — forma débil incremental

Principio de trabajos virtuales completo:

$$\underbrace{\int_V \delta \varepsilon \sigma dV}_{\delta W_{\text{int}}} = \underbrace{\int_V \delta \mathbf{u}^\top \mathbf{b} dV + \int_{\partial V_t} \delta \mathbf{u}^\top \bar{\mathbf{t}} dA}_{\delta W_{\text{ext}}}.$$

El elemento calcula **solo el lado interno**:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}},$$

con la rigidez tangente consistente separada en parte material y geométrica (ver §9). Las cargas externas (nodales y, si las hubiera, de cuerpo o tracción) las ensambla el solver a partir del input del usuario; este elemento **no** produce vector nodal equivalente de fuerzas de cuerpo distribuidas a lo largo del eje.

1.9 Formulación numérica (FEM)

1.9.1 6. Discretización

Dos nodos, 2 DOFs por nodo (u_x, u_y) . Vector elemental $\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top$.

1.9.2 7. Vector dirección actual

$$\mathbf{d} = [-c_\theta, -s_\theta, c_\theta, s_\theta]^\top, \quad \mathbf{n} = [-s_\theta, c_\theta, s_\theta, -c_\theta]^\top,$$

con c_θ, s_θ evaluados en la configuración corriente.

1.9.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.9.4 9. Rigidez tangente

$$\begin{aligned} \mathbf{K}_T &= \mathbf{K}_M + \mathbf{K}_G, \\ \mathbf{K}_M &= \frac{EA}{L_0} \mathbf{d} \mathbf{d}^\top, \\ \mathbf{K}_G &= \frac{N}{l} \mathbf{n} \mathbf{n}^\top, \quad N = \sigma A. \end{aligned}$$

1.9.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

1.9.6 11. Cuadratura

No aplica (forma cerrada; un único punto conceptual en el eje).

1.10 Contrato de implementación

```
name: Truss2DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: Área de la sección transversal }
```

```

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: axial escalar (deformación ingenieril corotacional)

conventions:
  sign: " > 0 > 0 (elongación)"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_T y F_int invariantes bajo permutación"
  configuration: "Updated Lagrangian - c_, s_, l se recalculan en cada evaluación"

validity:
  - "|| 1e-2 (pequeña deformación axial)"
  - rotaciones y desplazamientos de cualquier magnitud
  - cargas exclusivamente axiales

out_of_scope:
  - flexión, cortante, momentos
  - pandeo por bifurcación (captura de snap-through requiere arc-length)
  - plasticidad con grandes deformaciones
  - "fuerzas de cuerpo distribuidas (peso propio sobre la barra): el elemento no
    calcula vector nodal equivalente; el usuario reparte masa a los nodos en el input"

acceptance:
  - name: equivalencia con Truss2D en régimen lineal
    setup: "barra axial empotrada, carga F pequeña ( ~ 1e-6)"
    expect: "u(L) = F·L/(E·A) con tol_rel 1e-8"
    tol_rel: 1.0e-8

  - name: invariancia bajo rotación de cuerpo rígido
    setup: "aplicar rotación rígida de 45° a ambos nodos (sin deformar)"
    expect: "F_int = 0 y = 0"
    tol_abs: 1.0e-10

  - name: rigidez geométrica bajo tensión
    setup: "barra en tensión con N > 0; verificar autovalor transversal"
    expect: "autovalor de K_T en dirección n = N/l"
    tol_rel: 1.0e-10

references:
  - "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
    .1, §3.3"
  - "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"

```

1.11 Implementación

- **Archivo:** fenix/elements/truss.py
- **Clase:** Truss2DCorot — hereda de Truss2D (reutiliza `__init__`, `ClassVars` y metadatos de registro) y sobrescribe `compute_element_state` y `compute_internal_forces` para evaluar cosenos directores y longitud en configuración corriente.
- **Tests:** tests/test_truss.py · TestTruss2DCorot — tres tests, uno por cada acceptance:
- test_acceptance_linear_limit_matches_truss2d (criterio 1).
- test_acceptance_rigid_body_rotation (criterio 2).

- `test_acceptance_geometric_stiffness_under_traction` (criterio 3).
- **Notas de traducción:**
- La longitud corriente l y los cosenos c_θ, s_θ se recalculan en cada evaluación (no se cachean entre llamadas del solver, como exige Updated Lagrangian).
- El vector $\mathbf{d}$ de §7 se construye con signos $[-c_\theta, -s_\theta, c_\theta, s_\theta]$; el transverso $\mathbf{n}$ como $[-s_\theta, c_\theta, s_\theta, -c_\theta]$. Ambos con norma $\sqrt{2}$ — el factor se absorbe en los coeficientes EA/L_0 y N/l .
- La rigidez tangente se devuelve ya sumada $\mathbf{K}_M + \mathbf{K}_G$; el solver no distingue las dos contribuciones.
- El test 3 confronta $\mathbf{K}_T$ con la descomposición esperada y además verifica $\mathbf{K}_G \mathbf{n} = (2N/l)\mathbf{n}$ (autovalor sobre el vector sin normalizar, que es N/l sobre el versor — así queda la ambigüedad resuelta explícitamente).

1.12 Diálogo

- **2026-04-21** · Implementación inicial tras validar el diseño spec-first. Se optó por herencia `Truss2DCorot(Truss2D)` para reutilizar el constructor (L_0 , cosenos iniciales) y los `ClassVars` del registro; los métodos que dependen de configuración corriente se sobrescriben. La bandera `large_strains` que tenía `Truss2D` previamente se elimina en el mismo commit — el régimen corotacional vive ahora como elemento independiente.
- **2026-04-21** · Normalización del autovalor transverso (criterio 3): la fórmula del libro da N/l sobre el **versor** $\hat{\mathbf{n}} = \mathbf{n}/\sqrt{2}$; sobre el vector $\mathbf{n}$ directo (norma $\sqrt{2}$) el autovalor es $2N/l$. El test verifica la segunda forma para evitar raíces cuadradas y la nota de traducción documenta la equivalencia.

1.13 Truss3D

*Orden de trabajo. El usuario escribe la **especificación física**, la **formulación numérica** y el **contrato**. La IA rellena **implementación** y responde en **diálogo** durante el trabajo.*

1.14 Especificación física

1.14.1 0. Descripción general

Elemento sólido 1D de primer orden, inmerso en el espacio tridimensional. Dos nodos articulados; transmite únicamente esfuerzo axial. Aproximación C^0 del desplazamiento. Régimen estrictamente de **linealidad geométrica**: pequeños desplazamientos, pequeñas rotaciones, pequeñas deformaciones.

1.14.2 1. Cinemática de desplazamientos

Interpolación lineal del desplazamiento axial a lo largo del eje $s \in [0, L]$:

$$u(s) = a_0 + a_1 s.$$

1.14.3 2. Cinemática de deformaciones

Única componente (axial, en el sistema local de la barra):

$$\varepsilon_{ss}(s) = \frac{du}{ds}.$$

1.14.4 3. Ecuación constitutiva

$$\sigma_{ss} = E \varepsilon_{ss}.$$

El elemento admite cualquier material 1D con STRAIN_DIM = 1 (elástico lineal, plástico 1D, daño 1D).

1.14.5 4. Equilibrio — forma fuerte

$$-\frac{d}{ds} \left(EA \frac{du}{ds} \right) = b(s), \quad s \in (0, L),$$

con condiciones de contorno Dirichlet o Neumann ($\sigma A = \bar{F}$) en los extremos.

1.14.6 5. Equilibrio — forma débil

$$\int_0^L \frac{d\delta u}{ds} EA \frac{du}{ds} ds = \int_0^L \delta u b ds + [\delta u \bar{F}]_{\partial\Omega}, \quad \forall \delta u \in V_0.$$

El elemento calcula **solo el lado interno**; las cargas externas las ensambla el solver a partir del input del usuario.

1.15 Formulación numérica (FEM)

1.15.1 6. Discretización

Dos nodos en $s = 0$ y $s = L$. Cosenos directores del eje en coordenadas globales:

$$c_x = \frac{x_2 - x_1}{L}, \quad c_y = \frac{y_2 - y_1}{L}, \quad c_z = \frac{z_2 - z_1}{L},$$

con $L = \|\mathbf{X}_2 - \mathbf{X}_1\|$ (configuración inicial, fija).

1.15.2 7. Funciones de forma

$$N_1(s) = 1 - \frac{s}{L}, \quad N_2(s) = \frac{s}{L}.$$

1.15.3 8. Matriz deformación-desplazamiento

Local (1 DOF axial por nodo): $\mathbf{B}_{\text{loc}} = \frac{1}{L}[-1, 1]$. Global con $\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top$:

$$\mathbf{B} = \frac{1}{L}[-c_x, -c_y, -c_z, c_x, c_y, c_z], \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.15.4 9. Rigidez elemental

Integración analítica exacta (EA constante, $\mathbf{B}$ constante):

$$\mathbf{K}_e = \int_0^L \mathbf{B}^\top EA \mathbf{B} ds = \frac{EA}{L} \mathbf{d} \mathbf{d}^\top, \quad \mathbf{d} = [-c_x, -c_y, -c_z, c_x, c_y, c_z]^\top.$$

$\mathbf{K}_e$ es una matriz 6×6 de rango 1.

1.15.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sigma A \mathbf{d} = N \mathbf{d}.$$

1.15.6 11. Cuadratura

No aplica (forma cerrada, $\mathbf{B}$ y σ uniformes por elemento). El campo `n_integration_points: 1` se refiere al número de estados materiales conceptuales (uno por elemento), no a puntos de Gauss.

1.16 Contrato de implementación

```
name: Truss3D
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1
```

```

parameters:
- { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
signature: "compute_state(, state) -> (, E_tangent, state)"
strain_kind: axial escalar

conventions:
sign: " > 0 > 0 (elongación)"
voigt: "N/A (escalar)"
node_orientation: "libre; K_e y F_int invariantes bajo permutación"
configuration: "inicial fija - L, c_x, c_y, c_z no se actualizan con los
desplazamientos"

validity:
- "|| 1e-2"
- pequeños desplazamientos y rotaciones
- cargas exclusivamente axiales

out_of_scope:
- flexión, cortante, momentos
- grandes desplazamientos o rotaciones (para ese régimen usar `Truss3DCorot`)
- pandeo
- "fuerzas de cuerpo distribuidas sobre el eje: el usuario reparte masa a los nodos"

acceptance:
- name: barra axial bajo carga puntual (3D)
setup: "empotrada en s=0, carga F axial en s=L, orientación arbitraria en el
espacio"
expect: "u(L) = F·L / (E·A) proyectado sobre el eje"
tol_rel: 1.0e-10

references:
- "Bathe K.-J., Finite Element Procedures, §4.2.1"
- "Cook R.D., Concepts and Applications of FEA, §2.3"

```

1.17 Implementación

- **Archivo:** fenix/elements/truss.py
- **Clase:** Truss3D — hereda directamente de Element (sin relación de herencia con Truss2D ni con Truss2DCorot).
- **Tests:** tests/test_truss.py · TestTruss3D — verifica L0, cosenos directores (c_x, c_y, c_z), entradas de $\mathbf{K}_e$ (incluyendo simetría, dimensiones 6×6) y evaluación de $\mathbf{F}_{\text{int}}$ sobre desplazamientos conocidos con geometría de prueba $L = 13$, $(c_x, c_y, c_z) = (3/13, 4/13, 12/13)$.
- **Notas de traducción:**
 - L0, cx, cy, cz se calculan una vez en `__init__` sobre la configuración inicial y no se actualizan — coherente con el régimen geoméricamente lineal declarado.
 - El constructor tolera nodos con 2 ó 3 coordenadas (completa con $z = 0$ si falta), para facilitar transición de problemas planos embebidos.

- El contrato con el material es el estándar del proyecto: `material.compute_state( $\varepsilon$ , state)  $\rightarrow$  ( $\sigma$ ,  $E_t$ , state')`.
- Para grandes desplazamientos/rotaciones en 3D usar `Truss3DCorot`, que hereda de esta clase.

1.18 Diálogo

- **2026-04-21** · Apertura de spec retroactiva. La clase `Truss3D` preexistía al flujo spec-first; la spec se crea documentando la formulación ya implementada y estableciendo explícitamente su alcance: régimen de linealidad geométrica, sin bandera ni variante corotacional. Los tests unitarios existentes cubren el criterio de **acceptance** (vía los valores de $\mathbf{K}_e = (EA/L)\mathbf{d}\mathbf{d}^\top$ que implican algebraicamente $u(L) = FL/(EA)$). Promovido **status: validated**.

1.19 Truss3DCorot

*Orden de trabajo. Usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

1.20 Especificación física

1.20.1 0. Descripción general

Barra articulada 3D equivalente a Truss3D pero en régimen de **grandes desplazamientos y rotaciones** con **pequeña deformación axial**. Updated Lagrangian / corotacional: el eje de la barra se redefine en cada iteración sobre la configuración corriente y la deformación se mide sobre la longitud actual. La rigidez tangente suma material + geométrica.

1.20.2 1. Cinemática

Configuraciones:

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^3$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Vector unitario del eje corriente:

$$\hat{\mathbf{e}} = \frac{\mathbf{x}_2 - \mathbf{x}_1}{l} = (c_x, c_y, c_z), \quad c_x^2 + c_y^2 + c_z^2 = 1.$$

1.20.3 2. Medida de deformación

Ingenieril corotacional:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

1.20.4 3. Ecuación constitutiva

Material 1D con STRAIN_DIM = 1: $\sigma = \sigma(\varepsilon)$ (el elemento no hardcodea la ley; la provee el material).

1.20.5 4. Equilibrio — forma débil incremental

PTV estándar. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

1.21 Formulación numérica (FEM)

1.21.1 6. Discretización

Dos nodos, 3 DOFs por nodo (u_x, u_y, u_z). Vector elemental de 6 componentes:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top.$$

1.21.2 7. Vector dirección actual

$$\mathbf{d} = [-c_x, -c_y, -c_z, c_x, c_y, c_z]^\top,$$

con los cosenos evaluados en configuración **corriente**. $\|\mathbf{d}\|^2 = 2$.

1.21.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.21.4 9. Rigidez tangente

Material:

$$\mathbf{K}_M = \frac{E_t A}{L_0} \mathbf{d} \mathbf{d}^\top \quad (6 \times 6, \text{ rango } 1).$$

Geométrica: en 3D la dirección perpendicular al eje es un **plano** (no un vector único). Se usa el proyector perpendicular $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ (matriz 3×3 , rango 2). La rigidez geométrica en el elemento de 6 DOFs:

$$\mathbf{K}_G = \frac{N}{l} \begin{bmatrix} \mathbf{P}_3 & -\mathbf{P}_3 \\ -\mathbf{P}_3 & \mathbf{P}_3 \end{bmatrix} \quad (6 \times 6, \text{ rango } 2), \quad N = \sigma A.$$

1.21.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

1.21.6 11. Cuadratura

No aplica (forma cerrada).

1.22 Contrato de implementación

```
name: Truss3DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state)"
  strain_kind: axial escalar (deformación ingenieril corotacional)

conventions:
```

```

sign: " > 0    > 0 (elongación)"
voigt: "N/A (escalar)"
node_orientation: "libre"
configuration: "Updated Lagrangian - l, c_x, c_y, c_z se recalculan en cada evaluación"

validity:
- "|| 1e-2"
- rotaciones y desplazamientos de cualquier magnitud en el espacio
- cargas exclusivamente axiales

out_of_scope:
- flexión, cortante, momentos
- pandeo por bifurcación
- plasticidad con grandes deformaciones
- "fuerzas de cuerpo distribuidas: el usuario reparte masa a los nodos"

acceptance:
- name: equivalencia con Truss3D en régimen lineal
  setup: "barra 3D con orientación arbitraria, carga axial pequeña ( ~ 1e-6)"
  expect: "K_T y F_int coinciden con Truss3D lineal a tolerancia rtol=1e-4"
  tol_rel: 1.0e-4

- name: invariancia bajo rotación rígida 3D
  setup: "aplicar una rotación rígida arbitraria (p. ej. 30° sobre eje z, luego 45° sobre eje x)"
  expect: "F_int = 0 y    = 0"
  tol_abs: 1.0e-10

- name: rigidez geométrica transversa (plano perpendicular)
  setup: "barra en tensión con N > 0; aplicar K_G sobre dos vectores perpendiculares al eje linealmente independientes"
  expect: "K_G·v_perp = (N/l)·v_perp_struct, donde v_perp_struct es la extensión del vector transverso al espacio de 6 DOFs con signos opuestos en los dos nodos"
  tol_rel: 1.0e-10

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol .1, §3.3"
- "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"

```

1.23 Implementación

- **Archivo:** fenix/elements/truss.py
- **Clase:** Truss3DCorot — hereda de Truss3D (reutiliza `__init__`, tolerancia a nodos 2D/3D, metadatos de registro) y sobrescribe `compute_element_state` y `compute_internal_forces` para evaluar longitud y cosenos directores en configuración corriente.
- **Tests:** tests/test_truss.py · TestTruss3DCorot — tres tests, uno por cada acceptance:
- test_acceptance_linear_limit_matches_truss3d (criterio 1).
- test_acceptance_rigid_body_rotation (criterio 2 — dos rotaciones rígidas distintas, verifica $\sigma=0$ y $F_{int}=0$ en ambas).

- `test_acceptance_geometric_stiffness_transverse_plane` (criterio 3 — aplica $\mathbf{K}_G$ a **dos** direcciones linealmente independientes del plano perpendicular al eje).
- **Notas de traducción:**
- La clave respecto a 2D es el proyector $\mathbf{P}_3 = \mathbf{I} - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ (3×3 , rango 2): el plano perpendicular al eje es bidimensional. $\mathbf{K}_G$ resulta entonces de rango 2, no rango 1.
- Método estático `_perpendicular_projector(cx, cy, cz)` extrae esta operación para legibilidad y posible reuso.
- La longitud corriente l y los cosenos c_x, c_y, c_z se recalculan en cada evaluación (Updated Lagrangian); no se cachean entre llamadas del solver.
- Autovalores de $\mathbf{K}_G$ sobre modos transversos (rotación de la barra alrededor de su centro): $2N/l$ sobre el vector sin normalizar — igual que en 2D, el factor 2 viene de que el vector de 6 componentes tiene norma $\sqrt{2}$. Sobre el versor el autovalor es N/l .

1.24 Diálogo

- **2026-04-21** · Implementación tras validar el diseño spec-first. Se optó por herencia `Truss3DCorot(Truss3D)` por paralelismo con `Truss2DCorot(Truss2D)` y reutilización del constructor. La diferencia estructural con el caso 2D es el proyector $\mathbf{P}$: en 2D tenía rango 1 y generaba un $\mathbf{K}_G$ de rango 1 (vector $\mathbf{n}$ único); en 3D tiene rango 2 y $\mathbf{K}_G$ rango 2 (plano perpendicular al eje). Los tests del criterio 3 verifican explícitamente **dos** direcciones transversas linealmente independientes, capturando este cambio dimensional.
- **2026-04-21** · Test de invariancia bajo rotación rígida 3D: se verifican dos rotaciones distintas (plano x-y y plano x-z) para descartar coincidencias accidentales en ejes canónicos.

Capítulo 2

Elementos 1D — Cables

2.1 Cable2DCorot

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

2.2 Especificación física

2.2.1 0. Descripción general

Elemento 1D inmerso en el plano 2D que modela un **cable**: dos nodos articulados que solo transmiten **esfuerzo axial de tensión**. En compresión o estado sin tensar, el elemento no transmite nada — se destensa. Cinemática **corotacional** (Updated Lagrangian): el eje rota con el movimiento, la longitud se mide sobre la configuración corriente.

La propiedad de unilateralidad vive en el **material**; el elemento es la maquinaria cinemática que aplica al material la deformación correcta. Para obtener el comportamiento propio de cable se empareja este elemento con un material unilateral como **CableMaterial1D**. Con un material elástico clásico, el elemento se comporta como una barra articulada corotacional (útil para pruebas, no para modelado realista de cables).

2.2.2 1. Cinemática

Configuraciones:

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^2$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Cosenos directores del eje **corriente**:

$$c_\theta = \frac{x_2 - x_1}{l}, \quad s_\theta = \frac{y_2 - y_1}{l}.$$

2.2.3 2. Medida de deformación

Ingenieril corotacional:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

2.2.4 3. Ecuación constitutiva

Delegada al material con `STRAIN_DIM = 1`. El elemento invoca `material.compute_state(ε, state)` y recibe $(\sigma, E_t, \text{nuevo estado})$ sin inspeccionar su naturaleza. La unilateralidad (si la hay) es responsabilidad del material — el elemento nunca filtra σ ni ε por signo.

2.2.5 4. Equilibrio — forma débil incremental

Principio de trabajos virtuales, lado interno:

$$\delta W_{\text{int}} = \int_V \delta \varepsilon \sigma dV = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

Las cargas externas las ensambla el solver. El elemento no calcula vector nodal equivalente de fuerzas de cuerpo.

2.3 Formulación numérica (FEM)

2.3.1 6. Discretización

Dos nodos, 2 DOFs por nodo (u_x, u_y) . Vector elemental:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top.$$

2.3.2 7. Vectores de dirección (configuración corriente)

$$\mathbf{d} = [-c_\theta, -s_\theta, c_\theta, s_\theta]^\top, \quad \mathbf{n} = [-s_\theta, c_\theta, s_\theta, -c_\theta]^\top.$$

$\mathbf{d}$ apunta a lo largo del eje corriente (norma $\sqrt{2}$); $\mathbf{n}$ al perpendicular (norma $\sqrt{2}$). Son ortogonales: $\mathbf{d}^\top \mathbf{n} = 0$.

2.3.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

2.3.4 9. Rigidez tangente

$$\mathbf{K}_T = \mathbf{K}_M + \mathbf{K}_G,$$

$$\mathbf{K}_M = \frac{E_t A}{L_0} \mathbf{d} \mathbf{d}^\top, \quad \mathbf{K}_G = \frac{N}{l} \mathbf{n} \mathbf{n}^\top, \quad N = \sigma A.$$

Caso destensado (material unilateral con $\varepsilon \leq 0$): el material devuelve $\sigma = 0$, $E_t = 0$, luego $N = 0$ y **ambas contribuciones se anulan** — $\mathbf{K}_T = \mathbf{0}$. El elemento aporta cero al sistema global, como físicamente corresponde a un cable flojo.

2.3.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

Cuando el cable está destensado ($N = 0$), $\mathbf{F}_{\text{int}} = \mathbf{0}$.

2.3.6 11. Cuadratura

No aplica (forma cerrada).

2.4 Contrato de implementación

```

name: Cable2DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal del cable" }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state)"
  strain_kind: axial escalar (deformación ingenieril corotacional)
  expected_behaviour: "material unilateral (p. ej. CableMaterial1D) para obtener respuesta física de cable; cualquier material con STRAIN_DIM=1 es técnicamente aceptado pero producirá comportamiento de barra"

conventions:
  sign: " > 0 > 0 (elongación); la responsabilidad de anular en compresión es del material"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_T y F_int invariantes bajo permutación"
  configuration: "Updated Lagrangian - l, c_, s_ se recalculan en cada evaluación"

validity:
  - "|| 1e-2 en régimen tensado"
  - rotaciones y desplazamientos de cualquier magnitud
  - cargas axiales (las transversales se transmiten por el destensado-retensado del cable)

out_of_scope:
  - flexión, cortante, momentos
  - dinámica con inercia distribuida (sin matriz de masa)
  - pandeo (irrelevante: el cable no transmite compresión)
  - "fuerzas de cuerpo distribuidas: el usuario reparte masa/peso a los nodos"
  - pretensión mediante parámetro del elemento (modelar con longitud inicial ajustada)

numerical_caveats:

```

```

- "Un cable completamente destensado aporta  $K_T = 0$  al sistema global. Si otros
  elementos no garantizan rigidez suficiente en los DOFs compartidos, la matriz
  tangente global puede ser singular o mal condicionada."
- "En transiciones tensado destensado ( cruza 0), Newton-Raphson puede oscilar por
  el salto discontinuo en  $E_t$ . Usar arc-length o pasos de carga finos si el
  problema atraviesa destensiones."

acceptance:
- name: cable tensado coincide con barra corotacional
  setup: "cable horizontal con material lineal (Elastic1D,  $E=1000$ ),  $u_x$  nodo 2 =  $1e-3$ 
         =  $1e-3 > 0$ "
  expect: " =  $E$  ,  $E_t = E$ ,  $K_T = K_M + K_G$  con valores de barra corotacional"
  tol_rel: 1.0e-10

- name: cable destensado produce aportación nula
  setup: "cable horizontal con CableMaterial1D,  $u_x$  nodo 2 =  $-1e-3$  < 0"
  expect: " = 0,  $F_{int} = 0$ ,  $K_T =$  matriz cero"
  tol_abs: 1.0e-12

- name: invariancia bajo rotación rígida
  setup: "cable inicialmente horizontal, rotación rígida de  $45^\circ$  de ambos nodos"
  expect: " = 0 y  $F_{int} = 0$  (independiente del material)"
  tol_abs: 1.0e-10

- name: cruce por cero
  setup: " $u_e$  tal que = 0 exactamente, material unilateral"
  expect: " = 0,  $E_t = 0$ ,  $F_{int} = 0$ ,  $K_T = 0$ "
  tol_abs: 1.0e-12

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
  .1, §3.3"
- "Irvine H.M., Cable Structures, MIT Press, §1.2"

```

2.5 Implementación

- **Archivo:** `fenix/elements/cable.py` — archivo propio, no referencia a los módulos de armadura ni a ningún otro elemento.
- **Clase:** `Cable2DCorot` — hereda directamente de `Element` (no de `Truss2DCorot` ni de ningún elemento de armadura). La maquinaria cinemática corotacional se reimplementa íntegra dentro de la clase.
- **Tests:** `tests/test_cable_elements.py · TestCable2DCorot` — los cuatro criterios de `acceptance` más un test de registro:
- `test_acceptance_cable_tensado_como_barra_corot` (criterio 1).
- `test_acceptance_cable_destensado_aportacion_nula` (criterio 2).
- `test_acceptance_rotacion_rigida` (criterio 3).
- `test_acceptance_cruce_por_cero` (criterio 4).
- `test_registro_en_registry` — autodiscover.

- **Notas de traducción:**

- La clase duplica deliberadamente la lógica de `Truss2DCorot`. No hay herencia. Es el precio de la independencia: si mañana se toca la armadura corotacional, este cable no se mueve.
- `_current_geometry(u_e)` devuelve $(1, c_\theta, s_\theta)$ — método privado para aislar la única operación que depende de configuración corriente.
- En estado destensado, $\mathbf{K}_M = \mathbf{0}$ (porque $E_t = 0$ viene del material) y $\mathbf{K}_G = \mathbf{0}$ (porque $N = 0$); el elemento aporta literalmente ceros al ensamblaje, como debe ser físicamente un cable flojo. Los tests lo verifican con tolerancia absoluta.
- El elemento no valida que el material sea unilateral: es responsabilidad del usuario emparejar con `CableMaterial1D`. Emparejar con `Elastic1D` produce respuesta de barra corotacional y se usa internamente en el test del criterio 1 para comparar contra valores analíticos.

2.6 Diálogo

- **2026-04-21** · Elemento creado como componente totalmente independiente de `Truss2DCorot` por decisión explícita del usuario: la maquinaria cinemática se duplica dentro de la clase de cable para desacoplar su evolución futura del elemento de armadura. El coste es 40 líneas duplicadas; la ganancia es que ninguna refactorización de las armaduras puede contaminar silenciosamente el comportamiento de los cables.
- **2026-04-21** · Validación de material unilateral: se optó por **no** imponerla en `__init__`. Motivo: el contrato elemento↔material del proyecto es genérico sobre `STRAIN_DIM=1`; introducir una verificación específica crearía un acoplamiento entre la clase del elemento y la identidad del material. Se documenta en `material_contract.expected_behaviour` del YAML para orientar al usuario.

2.7 Cable3DCorot

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

2.8 Especificación física

2.8.1 0. Descripción general

Elemento 1D inmerso en el espacio 3D que modela un **cable**: dos nodos articulados que solo transmiten **esfuerzo axial de tensión**. En compresión o estado sin tensar, el elemento no transmite nada. Cinemática **corotacional** (Updated Lagrangian): el eje rota libremente en el espacio, la longitud se mide sobre la configuración corriente.

La unilateralidad vive en el **material**; el elemento es la maquinaria cinemática 3D. Para obtener respuesta física de cable, emparejar con **CableMaterial1D**. Con un material elástico clásico, el elemento se comporta como una barra corotacional 3D.

2.8.2 1. Cinemática

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^3$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Vector unitario del eje corriente:

$$\hat{\mathbf{e}} = \frac{\mathbf{x}_2 - \mathbf{x}_1}{l} = (c_x, c_y, c_z), \quad c_x^2 + c_y^2 + c_z^2 = 1.$$

2.8.3 2. Medida de deformación

Ingenieril corotacional:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

2.8.4 3. Ecuación constitutiva

Delegada al material con `STRAIN_DIM = 1`. El elemento llama `material.compute_state( $\varepsilon$ , state)` y recibe $(\sigma, E_t, \text{nuevo estado})$ sin inspeccionar su naturaleza. La unilateralidad (si la hay) es responsabilidad del material.

2.8.5 4. Equilibrio — forma débil incremental

PTV estándar; el elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^T \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

2.9 Formulación numérica (FEM)

2.9.1 6. Discretización

Dos nodos, 3 DOFs por nodo (u_x, u_y, u_z) . Vector elemental de 6 componentes:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top.$$

2.9.2 7. Vector dirección actual

$$\mathbf{d} = [-c_x, -c_y, -c_z, c_x, c_y, c_z]^\top, \quad \|\mathbf{d}\|^2 = 2,$$

con los cosenos evaluados en configuración **corriente**.

2.9.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

2.9.4 9. Rigidez tangente

Material:

$$\mathbf{K}_M = \frac{E_t A}{L_0} \mathbf{d} \mathbf{d}^\top \quad (6 \times 6, \text{ rango } 1).$$

Geométrica: en 3D la dirección perpendicular al eje es un plano bidimensional. Se usa el proyector perpendicular $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}} \hat{\mathbf{e}}^\top$ (3×3 , rango 2):

$$\mathbf{K}_G = \frac{N}{l} \begin{bmatrix} \mathbf{P}_3 & -\mathbf{P}_3 \\ -\mathbf{P}_3 & \mathbf{P}_3 \end{bmatrix} \quad (6 \times 6, \text{ rango } 2), \quad N = \sigma A.$$

Caso destensado (material unilateral con $\varepsilon \leq 0$): el material devuelve $\sigma = 0$, $E_t = 0$, luego $N = 0$ y **ambas contribuciones se anulan** — $\mathbf{K}_T = \mathbf{0}$. El elemento aporta cero al sistema global.

2.9.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

2.9.6 11. Cuadratura

No aplica (forma cerrada).

2.10 Contrato de implementación

```
name: Cable3DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
```

```

n_nodes: 2
strain_dim: 1
n_integration_points: 1

parameters:
- { name: A, type: float, required: true, desc: "Área de la sección transversal del cable" }

material_contract:
signature: "compute_state(, state) -> (, E_tangent, state)"
strain_kind: axial escalar (deformación ingenieril corotacional)
expected_behaviour: "material unilateral (p. ej. CableMaterial1D) para obtener respuesta física de cable; cualquier material con STRAIN_DIM=1 es técnicamente aceptado pero producirá comportamiento de barra"

conventions:
sign: " > 0 > 0 (elongación); la responsabilidad de anular en compresión es del material"
voigt: "N/A (escalar)"
node_orientation: "libre; K_T y F_int invariantes bajo permutación"
configuration: "Updated Lagrangian - l, c_x, c_y, c_z se recalculan en cada evaluación"

validity:
- "|| 1e-2 en régimen tensado"
- rotaciones y desplazamientos de cualquier magnitud en el espacio
- cargas axiales (las transversales se transmiten por destensado-retensado del cable)

out_of_scope:
- flexión, cortante, momentos, torsión
- dinámica con inercia distribuida (sin matriz de masa)
- pandeo (irrelevante: el cable no transmite compresión)
- "fuerzas de cuerpo distribuidas: el usuario reparte masa/peso a los nodos"
- pretensión mediante parámetro del elemento

numerical_caveats:
- "Cable destensado K_T = 0 en los DOFs del elemento. Si otros elementos no garantizan rigidez en esos DOFs, la matriz tangente global puede ser singular."
- "En transiciones tensado destensado (cruza 0), Newton-Raphson puede oscilar. Usar arc-length o pasos finos si el problema atraviesa destensiones."

acceptance:
- name: cable tensado coincide con barra corotacional 3D
  setup: "cable con orientación arbitraria, material lineal (Elastic1D), desplazamiento axial pequeño ( ~ 1e-6)"
  expect: "K_T y F_int coinciden con Truss3DCorot a tolerancia rtol = 1e-4"
  tol_rel: 1.0e-4

- name: cable destensado produce aportación nula
  setup: "cable sobre eje x con CableMaterial1D, u_x nodo 2 = -1e-3 < 0"
  expect: " = 0, F_int = 0, K_T = matriz cero"
  tol_abs: 1.0e-12

- name: invariancia bajo rotación rígida 3D
  setup: "cable inicial sobre eje x, rotación rígida de 45° en plano x-z"
  expect: " = 0 y F_int = 0 (independiente del material)"
  tol_abs: 1.0e-10

- name: cruce por cero

```



```

setup: "u_e = 0 con material unilateral"
expect: " = 0, E_t = 0, F_int = 0, K_T = 0"
tol_abs: 1.0e-12

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
  .1, §3.3"
- "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"
- "Irvine H.M., Cable Structures, MIT Press, §1.2"

```

2.11 Implementación

- **Archivo:** fenix/elements/cable.py — comparte archivo con `Cable2DCorot` por convención temática del proyecto; no comparte código.
- **Clase:** `Cable3DCorot` — hereda directamente de `Element` (no de `Cable2DCorot`, no de `Truss3DCorot`, no de ningún otro elemento). Maquinaria cinemática 3D autónoma.
- **Tests:** `tests/test_cable_elements.py · TestCable3DCorot` — los 4 criterios de `acceptance` más un test de registro:
 - `test_acceptance_cable_tensado_como_barra_corot_3d` (criterio 1).
 - `test_acceptance_cable_destensado_aportacion_nula` (criterio 2).
 - `test_acceptance_rotacion_rigida_3d` (criterio 3).
 - `test_acceptance_cruce_por_cero` (criterio 4).
 - `test_registro_en_registry` — autodiscover.
- **Notas de traducción:**
 - La clase duplica la maquinaria corotacional 3D de `Truss3DCorot`. Duplicación deliberada por la misma razón que en 2D: desacoplar la evolución futura de armaduras y cables.
 - Método estático `_perpendicular_projector(cx, cy, cz)` extrae la construcción del proyector $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ para legibilidad.
 - `_current_geometry` tolera nodos con 2 ó 3 coordenadas (completa con $z = 0$ si falta), heredando la flexibilidad de `Truss3D` para problemas embebidos.
 - En estado destensado, $\mathbf{K}_M = \mathbf{0}$ y $\mathbf{K}_G = \mathbf{0}$ simultáneamente; el elemento aporta literalmente ceros al ensamblaje.
 - El test del criterio 1 compara directamente contra `Truss3DCorot` con material `Elastic1D`: si ambos producen $\mathbf{K}$ y $\mathbf{F}_{\text{int}}$ iguales a tolerancia, la cinemática del cable y de la barra son consistentes entre sí en el régimen tensado.

2.12 Diálogo

- **2026-04-21** · Elemento creado como pieza totalmente autónoma por decisión del usuario: no hereda de `Cable2DCorot` ni de `Truss3DCorot`. Compartir archivo con `Cable2DCorot` (`fenix/elements/cable.py`) es convención temática del proyecto (elementos del mismo dominio conviven en un archivo, como las 4 armaduras y los 2 frames 2D en `structural.py`), no implica dependencia de código.
- **2026-04-21** · La única diferencia estructural con `Cable2DCorot` es el proyector 3D: en 2D la perpendicular es un vector único ($\mathbf{n}$); en 3D es un plano (dimensión 2). Esto se refleja en el rango de $\mathbf{K}_G$: 1 en 2D, 2 en 3D.

Capítulo 3

Elementos 1D — Marcos / Vigas

3.1 Frame2DEuler

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.2 Especificación física

3.2.1 0. Descripción general

Elemento 1D inmerso en el plano 2D que modela una **viga esbelta** según la teoría de **Euler-Bernoulli**. Dos nodos rígidamente conectados (transmiten momento), 3 grados de libertad por nodo (u_x, u_y, r_z) . Captura tres acciones internas: **esfuerzo axial**, **cortante transverso** y **momento flector**. Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones).

3.2.2 1. Hipótesis cinemática de Euler-Bernoulli

- Las secciones transversales se mantienen **planas** tras la deformación.
- Las secciones permanecen **perpendiculares al eje neutro deformado** (cizalladura transversal nula: $\gamma_{xy} = 0$).

Consecuencia: la rotación de la sección es igual a la pendiente de la elástica:

$$\theta(s) = \frac{dv}{ds}.$$

Válido solo para vigas **esbeltas** ($L/h \gtrsim 10$). En vigas peraltadas la cizalladura no es despreciable
→ Frame2DTimoshenko.

3.2.3 2. Campos de desplazamiento

- Axial: $u(s)$ a lo largo del eje local $s \in [0, L]$.
- Transverso: $v(s)$ perpendicular al eje local.
- Rotación de la sección: $\theta(s) = dv/ds$ (derivada de la flecha).

3.2.4 3. Medidas de deformación

- Axial: $\varepsilon_{axial}(s) = du/ds$ (constante para interpolación lineal en u).
- Curvatura: $\kappa(s) = d^2v/ds^2$.

La deformación axial en un punto genérico de la sección es $\varepsilon(s, y) = \varepsilon_{axial}(s) - y \kappa(s)$; al integrar sobre la sección se obtienen axial (N) y momento flector (M) desacoplados.

3.2.5 4. Ecuaciones constitutivas

- Axial: $N = EA \varepsilon_{axial}$ (o $N = A \sigma$ con σ del material).
- Flexión: $M = EI \kappa$.

El material delega la respuesta axial; la rigidez a flexión escala con el **módulo tangente** E_t devuelto por el material (aproximación: toda la sección entra en régimen no-elástico simultáneamente — no hay plasticidad distribuida).

3.2.6 5. Equilibrio — forma débil

Principio de trabajos virtuales. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N \delta \varepsilon_{axial} + M \delta \kappa) ds = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}.$$

3.3 Formulación numérica (FEM)

3.3.1 6. Discretización

Dos nodos ($s = 0$ y $s = L$), 3 DOFs por nodo en coordenadas globales:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, r_z^{(2)}]^\top.$$

3.3.2 7. Sistema local y transformación

Cosenos directores del eje (configuración inicial, fija): $c = (x_2 - x_1)/L$, $s_\theta = (y_2 - y_1)/L$. Matriz de transformación ortogonal 6×6 :

$$\mathbf{T} = \begin{bmatrix} c & s_\theta & 0 & 0 & 0 & 0 \\ -s_\theta & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s_\theta & 0 \\ 0 & 0 & 0 & -s_\theta & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Las rotaciones r_z son invariantes ante rotación del sistema (escalares en el plano). $\mathbf{u}_{\text{local}} = \mathbf{T} \mathbf{u}_e$.

3.3.3 8. Funciones de forma

- Axial: lineal, $N_1^u(s) = 1 - s/L$, $N_2^u(s) = s/L$.
- Transverso (Hermite cúbicos, cuatro funciones que interpolan v y θ):

$$\begin{aligned} N_1^v &= 1 - 3\xi^2 + 2\xi^3, & N_1^\theta &= L(\xi - 2\xi^2 + \xi^3), \\ N_2^v &= 3\xi^2 - 2\xi^3, & N_2^\theta &= L(-\xi^2 + \xi^3), \end{aligned}$$

con $\xi = s/L$.

3.3.4 9. Rigidez local

En el sistema local, la matriz 6×6 tiene forma cerrada por integración analítica:

$$\mathbf{K}_{\text{local}} = \begin{bmatrix} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{12EI}{L^3} & \frac{6EI}{L^2} & 0 & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{4EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{2EI}{L} \\ -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{12EI}{L^3} & -\frac{6EI}{L^2} & 0 & \frac{12EI}{L^3} & -\frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{2EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{4EI}{L} \end{bmatrix}.$$

En no-linealidad material, $E \rightarrow E_t(\varepsilon_{axial})$ devuelto por el material — escala **todos** los términos de la matriz por igual. Esta es una aproximación: supone que la plasticidad/daño afecta uniformemente a la sección.

3.3.5 10. Fuerzas internas

En el sistema local:

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}} \mathbf{u}_{\text{local}}.$$

La componente axial se **corrige** con el esfuerzo axial verdadero del material: $F_1 = -\sigma A$, $F_4 = +\sigma A$. Esto permite que el material aporte σ no-lineal sin que la formulación de flexión interfiera.

3.3.6 11. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top \mathbf{K}_{\text{local}} \mathbf{T}, \quad \mathbf{F}_{\text{int}} = \mathbf{T}^\top \mathbf{F}_{\text{int,local}}.$$

3.3.7 12. Cuadratura

No aplica (forma cerrada para la matriz; integración analítica de los polinomios de Hermite). El campo `n_integration_points`: 1 se refiere al único estado material conceptual del elemento (la deformación axial media).

3.4 Contrato de implementación

```

name: Frame2DEuler
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: I, type: float, required: true, desc: "Momento de inercia respecto al eje perpendicular al plano (z)" }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar (deformación media axial = (u_2 - u_1)/L en sistema local)"
  nonlinearity_model: "E_tangent escala toda la matriz local - no hay distinción axial vs flexión en régimen no-elástico"

conventions:
  sign: "> 0 > 0 (elongación) en el eje axial"
  rotation_sign: "r_z positivo antihorario (convención matemática estándar)"
  node_orientation: "el eje local va del nodo 1 al nodo 2; permutar nodos invierte el signo de las fuerzas internas"
  configuration: "inicial fija - L, c, s, T no se actualizan con los desplazamientos"

validity:
  - "vigas esbeltas: L/h > 10 (h = canto de la sección)"
  - "pequeños desplazamientos y pequeñas rotaciones"
  - "|_axial| < 1e-2"

out_of_scope:
  - vigas peraltadas con deformación por cortante significativa (usar Frame2DTimoshenko)
  - grandes rotaciones o desplazamientos (no hay variante corotacional en el catálogo actual)
  - plasticidad distribuida en la sección (fibras); el modelo escala rigidez global por E_t
  - pandeo (requiere rigidez geométrica, no implementada en esta viga lineal)
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga equivalente a los nodos"

acceptance:
  - name: respuesta axial pura
    setup: "viga horizontal, empotrada en extremo izquierdo, carga axial F en extremo derecho"
    expect: "u_x (extremo libre) = F·L/(E·A); momentos y cortantes nulos"
    tol_rel: 1.0e-10

  - name: voladizo con carga transversal (flecha analítica)
    setup: "viga horizontal, empotrada en extremo izquierdo, carga vertical P en extremo derecho"
    expect: "v (extremo libre) = P·L³/(3·E·I); rotación = P·L²/(2·E·I)"
    tol_rel: 1.0e-10

```

```

- name: simetría de K
  setup: "cualquier configuración"
  expect: "K_global = K_global (elemento conservativo en régimen elástico)"
  tol_abs: 1.0e-12

references:
- "Bathe K.-J., Finite Element Procedures, §4.2 (formulación isoparamétrica) y cap. 5 (vigas)"
- "Cook R.D., Concepts and Applications of FEA, §2.7 (elementos de viga)"

```

3.5 Implementación

- **Archivo:** `fenix/elements/frame/euler.py` — submódulo del paquete `fenix/elements/frame/` que aloja también `Frame2DTimoshenko` y `Frame2DEulerCorot`.
- **Clase:** `Frame2DEuler` — hereda directamente de `Element`. La construcción de la matriz de transformación $\mathbf{T}$ se delega a `build_geometry_2d` en `fenix/elements/frame/_shared.py`, compartida con `Frame2DTimoshenko` (la versión corotacional reconstruye $\mathbf{T}$ desde `alpha0` por su lógica propia).
- **Tests:** `tests/test_frame.py · TestFrame2DEulerAcceptance` — los tres criterios físicos y el registro:
 - `test_acceptance_respuesta_axial_pura` (criterio 1) — resuelve el voladizo con `solver`, verifica $u_x(L) = FL/(EA)$.
 - `test_acceptance_voladizo_carga_transversal` (criterio 2) — carga transversal P , verifica flecha $v = PL^3/(3EI)$ y rotación $\theta = PL^2/(2EI)$.
 - `test_acceptance_simetria_K` (criterio 3) — viga oblicua ($c=0.6$, $s=0.8$) para evitar ejes canónicos, verifica $\mathbf{K} = \mathbf{K}^\top$.
- `test_registro_en_registry` — `autodiscover`.
- **Notas de traducción:**
 - La matriz $\mathbf{T}$ se calcula una sola vez en `__init__` sobre la configuración inicial y se guarda como atributo — coherente con el régimen geoméricamente lineal.
 - En `compute_element_state`, la componente axial de $\mathbf{F}_{\text{int,local}}$ se sobrescribe tras el producto $\mathbf{K}_{\text{local}}\mathbf{u}_{\text{local}}$ con el valor σA devuelto por el material. Esto permite usar materiales no-lineales en la rama axial sin que la parte lineal de flexión interfiera.
 - La aproximación del modelo no-lineal es que E_t escala **toda** la matriz local: no hay distinción entre rigidez axial y de flexión en régimen no-elástico. Documentado en `material_contract.nonlinearity_model`.

3.6 Diálogo

- **2026-04-21** · Elemento movido a archivo propio `fenix/elements/frame.py` y desacoplado del helper compartido `_frame_geometry`. El método estático `_build_geometry` reimplementa la construcción de $\mathbf{T}$ dentro de la clase. `Frame2DTimoshenko` sigue en `structural.py` usando su

propio acceso al helper compartido; si mañana también se independiza, duplicará o internalizará su propia versión.

- **2026-04-21** · Tests de aceptación cubren la flecha analítica del voladizo a 8 decimales ($v = PL^3/(3EI)$) usando el solver completo del proyecto — validan no solo la cinemática del elemento sino también su integración con ensamblador y solver.

- **2026-05-13** · `frame.py` se parte en paquete `fenix/elements/frame/{euler,timoshenko,euler_corot,_sh`

La duplicación de `_build_geometry` entre Euler y Timoshenko se elimina: la construcción de `T` vuelve a vivir en un único lugar (`build_geometry_2d` en `_shared.py`), pero esta vez sin función helper externa flotante — pertenece al paquete `frame/`. `Frame2DEulerCorot` mantiene su propia reconstrucción de `T` desde `alpha0` porque su cinemática corotacional no se beneficia del helper común.

3.7 Frame2DTimoshenko

*Orden de trabajo. El usuario escribe **especificación física, formulación numérica y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.8 Especificación física

3.8.1 0. Descripción general

Elemento 1D inmerso en el plano 2D que modela una **viga** según la teoría de **Timoshenko**. Dos nodos rígidamente conectados, 3 DOFs por nodo (u_x, u_y, r_z) . Transmite **esfuerzo axial, cortante transversal y momento flector**. Incluye explícitamente la **deformación por cortante transversal**. Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones).

3.8.2 1. Hipótesis cinemática de Timoshenko

- Las secciones transversales se mantienen **planas** tras la deformación (igual que Euler-Bernoulli).
- Las secciones **no** están obligadas a ser perpendiculares al eje neutro: se permite una **rotación adicional** respecto a la tangente.

Consecuencia: la rotación de la sección $\theta(s)$ y la pendiente de la elástica dv/ds son **variables independientes**:

$$\gamma(s) = \frac{dv}{ds} - \theta(s) \neq 0,$$

donde γ es la distorsión angular (deformación por cortante transversal). Euler-Bernoulli corresponde al caso $\gamma = 0$.

Apropiado para vigas **gruesas, cortas o peraltadas** ($L/h \lesssim 10$) donde la deformación por cortante no es despreciable.

3.8.3 2. Campos de desplazamiento

- Axial: $u(s)$ a lo largo del eje local $s \in [0, L]$.
- Transverso: $v(s)$ perpendicular al eje local.
- Rotación de la sección: $\theta(s)$, **independiente** de v .

3.8.4 3. Medidas de deformación

- Axial: $\varepsilon_{axial}(s) = du/ds$.
- Curvatura (flexión): $\kappa(s) = d\theta/ds$.
- Cortante: $\gamma(s) = dv/ds - \theta(s)$.

3.8.5 4. Ecuaciones constitutivas

- Axial: $N = EA \varepsilon_{axial}$ (el material entrega σ y E_t).
- Flexión: $M = EI \kappa$.
- Cortante: $V = GA_s \gamma$, con $G = E/[2(1 + \nu)]$ (módulo de corte) y A_s el **área efectiva de cortante** (menor que A por la distribución no uniforme de la tensión tangencial en la sección).

3.8.6 5. Factor de Phi (shear locking)

La formulación estándar con interpolaciones lineales de v y θ sufre **shear locking** para vigas esbeltas (la rigidez por cortante domina numéricamente aunque γ físicamente sea pequeño). La matriz local incorpora un factor correctivo:

$$\Phi = \frac{12 EI}{G A_s L^2}.$$

Para $\Phi \rightarrow 0$ (viga esbelta) la matriz se reduce a la de Euler-Bernoulli.

3.8.7 6. Equilibrio — forma débil

Principio de trabajos virtuales (PTV). El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N \delta \varepsilon_{axial} + M \delta \kappa + V \delta \gamma) ds = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}.$$

3.9 Formulación numérica (FEM)

3.9.1 7. Discretización

Dos nodos ($s = 0$ y $s = L$), 3 DOFs por nodo en coordenadas globales:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, r_z^{(2)}]^\top.$$

3.9.2 8. Sistema local y transformación

Cosenos directores del eje (configuración inicial, fija): $c = (x_2 - x_1)/L$, $s_\theta = (y_2 - y_1)/L$. Matriz de transformación ortogonal 6×6 :

$$\mathbf{T} = \begin{bmatrix} c & s_\theta & 0 & 0 & 0 & 0 \\ -s_\theta & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s_\theta & 0 \\ 0 & 0 & 0 & -s_\theta & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{u}_{\text{local}} = \mathbf{T} \mathbf{u}_e.$$

3.9.3 9. Rigidez local con corrección por shear locking

Coefficientes:

$$a = \frac{12 EI}{L^3(1 + \Phi)}, \quad b = \frac{6 EI}{L^2(1 + \Phi)}, \quad c_m = \frac{(4 + \Phi) EI}{L(1 + \Phi)}, \quad d = \frac{(2 - \Phi) EI}{L(1 + \Phi)}.$$

$$\mathbf{K}_{\text{local}} = \begin{bmatrix} EA/L & 0 & 0 & -EA/L & 0 & 0 \\ 0 & a & b & 0 & -a & b \\ 0 & b & c_m & 0 & -b & d \\ -EA/L & 0 & 0 & EA/L & 0 & 0 \\ 0 & -a & -b & 0 & a & -b \\ 0 & b & d & 0 & -b & c_m \end{bmatrix}.$$

Para $\Phi \rightarrow 0$: $a \rightarrow 12EI/L^3$, $b \rightarrow 6EI/L^2$, $c_m \rightarrow 4EI/L$, $d \rightarrow 2EI/L$ — se recupera Euler-Bernoulli.

3.9.4 10. Fuerzas internas

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}} \mathbf{u}_{\text{local}}.$$

La componente axial se corrige con el esfuerzo axial del material: $F_1 = -\sigma A$, $F_4 = +\sigma A$ (permite materiales no-lineales en la rama axial).

3.9.5 11. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top \mathbf{K}_{\text{local}} \mathbf{T}, \quad \mathbf{F}_{\text{int}} = \mathbf{T}^\top \mathbf{F}_{\text{int,local}}.$$

3.9.6 12. Cuadratura

No aplica (forma cerrada).

3.10 Contrato de implementación

```
name: Frame2DTimoshenko
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: I, type: float, required: true, desc: "Momento de inercia respecto al eje perpendicular al plano" }
  - { name: As, type: float, required: true, desc: "Área efectiva de cortante" }
  - { name: nu, type: float, required: false, default: 0.3, desc: "Coeficiente de Poisson (si el material no lo expone)" }
```

```

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar (deformación media = (u_2 - u_1)/L en sistema local)"
  nonlinearity_model: "E_tangent escala toda la matriz local (axial, flexión y cortante por igual)"
  poisson_source: "si material.nu existe, se usa; en su defecto el parámetro `nu` del elemento con default 0.3 y advertencia por consola"

conventions:
  sign: " > 0 > 0 (elongación) en el eje axial"
  rotation_sign: "r_z positivo antihorario"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "inicial fija - L, c, s, T,  $\phi$  (en forma funcional) se calculan sin actualizar"

validity:
  - "vigas peraltadas o cortas (L/h > 10) donde el cortante no es despreciable"
  - "pequeños desplazamientos y pequeñas rotaciones"
  - " $|_{axial}| < 1e-2$ "

out_of_scope:
  - "grandes rotaciones o desplazamientos (no hay variante corotacional en el catálogo actual)"
  - "plasticidad distribuida en la sección"
  - "pandeo"
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga equivalente a los nodos"

acceptance:
  - name: convergencia a Euler-Bernoulli para viga esbelta
    setup: "voladizo con carga transversal P, viga esbelta (L/h > 100)  $\phi \rightarrow 0$ "
    expect: " $v(L) = PL^3/(3EI)$  (solución Euler-Bernoulli) con tolerancia relativa  $1e-3$ "
    tol_rel: 1.0e-3

  - name: respuesta axial pura desacoplada
    setup: "voladizo con carga axial F"
    expect: " $u_x(L) = F \cdot L / (E \cdot A)$ ; momentos y cortantes nulos"
    tol_rel: 1.0e-10

  - name: simetría de K
    setup: "viga oblicua"
    expect: " $K_{global} = K_{global}$ "
    tol_abs: 1.0e-12

references:
  - "Reddy J.N., An Introduction to the Finite Element Method, cap. 5 (vigas de Timoshenko)"
  - "Bathe K.-J., Finite Element Procedures, §5.4 (formulación con factor  $\phi$ )"

```

3.11 Implementación

- **Archivo:** `fenix/elements/frame/timoshenko.py` — submódulo del paquete `fenix/elements/frame/` que aloja también `Frame2DEuler` y `Frame2DEulerCorot`.
- **Clase:** `Frame2DTimoshenko` — hereda directamente de `Element`, sin herencia con las otras vigas 2D. Comparte `build_geometry_2d` con `Frame2DEuler` en `fenix/elements/frame/_shared.py`;

además, los helpers libres de carga de cuerpo, masa consistente y traducción a `ElementForces` también viven en `_shared.py`.

- **Tests:** `tests/test_frame.py` · `TestFrame2DTimoshenkoAcceptance`:
- `test_acceptance_convergencia_euler_en_viga_esbelta` (criterio 1) — viga con L/h muy grande, verifica que la flecha tiende a $PL^3/(3EI)$ con tolerancia relativa 10^{-3} .
- `test_acceptance_respuesta_axial_pura` (criterio 2) — carga axial pura, verifica $u_x = FL/(EA)$.
- `test_acceptance_simetria_K` (criterio 3) — viga oblicua, verifica $\mathbf{K} = \mathbf{K}^\top$.
- `test_registro_en_registry` — autodiscover.
- **Notas de traducción:**
- Para no colisionar con la variable cinemática c (coseno director) dentro de `compute_element_state`, el coeficiente c_m de la matriz local se llama `c_coef` en el código y `d_coef` su análogo (antes eran `c` y `d` inline, ambiguos).
- El Poisson ν se toma de `material.nu` si el material lo expone; en su defecto, del parámetro `nu` del elemento con default 0.3 y una advertencia por consola la primera vez que se construye con ese default — atajo a errores silenciosos de usuarios que olvidan especificarlo.
- La matriz $\mathbf{T}$ se calcula una sola vez en `__init__` y se guarda como atributo (régimen geométricamente lineal).
- El factor Φ se recalcula en cada llamada porque depende de E_t devuelto por el material (en régimen no-lineal Φ cambia con el estado).

3.12 Diálogo

- **2026-04-21** · Elemento movido a archivo propio `fenix/elements/frame.py` y desacoplado del helper `_frame_geometry`. Con este movimiento, el helper compartido se elimina completamente del repositorio: `Frame2DEuler` y `Frame2DTimoshenko` replican la construcción de $\mathbf{T}$ como método estático `_build_geometry`, cada una en su clase. La duplicación (15 líneas) es el precio aceptado de la independencia mutua.
- **2026-04-21** · Test del criterio 1: se eligió L/h grande en lugar de reproducir un caso específico de libro porque la convergencia Timoshenko $\rightarrow$ Euler es el comportamiento **esperado por construcción** (factor $\Phi \rightarrow 0$). Verificarlo directamente con el solver completo valida simultáneamente la cinemática de Timoshenko, el tratamiento del shear locking y la integración con ensamblador + solver.
- **2026-05-13** · `frame.py` se parte en paquete `fenix/elements/frame/`. La duplicación de `_build_geometry` ya no se justifica: ambas vigas comparten `build_geometry_2d` en `_shared.py`, helper interno al paquete (no flotante). Las clases siguen sin herencia entre sí.

3.13 Frame2DEulerCorot

*Orden de trabajo. El usuario escribe **especificación física, formulación numérica y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.14 Especificación física

3.14.1 0. Descripción general

Viga 2D esbelta según Euler-Bernoulli en formulación **corotacional** (Updated Lagrangian). Dos nodos rígidamente conectados, 3 DOFs por nodo (u_x, u_y, r_z). Captura grandes desplazamientos y grandes rotaciones del elemento como un todo con **pequeñas deformaciones axiales** ($|\varepsilon_{axial}| \lesssim 1\%$) y rotaciones nodales deformacionales moderadas.

Abre dominios que la viga lineal no captura: **pandeo por flexión, post-pandeo, snap-through** de arcos planos, brazos flexibles esbeltos.

3.14.2 1. Cinemática corotacional

Se separa el movimiento del elemento en:

1. **Rotación rígida** α_e : el ángulo que el eje de la barra ha girado desde la configuración inicial.
2. **Rotación deformacional de cada sección** $\bar{\theta}_i$: la rotación nodal menos la rotación rígida (es lo que genera momento flector).

Sean:

$$\alpha_0 = \arctan 2(Y_2 - Y_1, X_2 - X_1) \quad (\text{ángulo del eje en config. inicial, fijo}),$$

$$\alpha = \arctan 2(y_2 - y_1, x_2 - x_1) \quad (\text{ángulo corriente}),$$

$$\alpha_e = \alpha - \alpha_0 \quad (\text{rotación rígida del elemento}).$$

Rotaciones deformacionales:

$$\bar{\theta}_1 = \theta_1 - \alpha_e, \quad \bar{\theta}_2 = \theta_2 - \alpha_e,$$

donde $\theta_i = r_z^{(i)}$ son las rotaciones totales de los nodos.

3.14.3 2. Medidas de deformación

- Axial: $\varepsilon_{axial} = (l - L_0)/L_0$ con l longitud corriente.
- Curvatura efectiva: $\kappa = (\bar{\theta}_2 - \bar{\theta}_1)/L_0$ (constante si se usa interpolación cúbica).

3.14.4 3. Ecuaciones constitutivas

- Axial: delegado al material (σ, E_t) .
- Flexión: $M = E_t I \kappa$ (el mismo E_t del material escala la rigidez de flexión, como en **Frame2DEuler**).

3.14.5 4. Equilibrio — forma débil

PTV en configuración corriente. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \mathbf{K}_{\text{material}} + \mathbf{K}_\sigma.$$

3.15 Formulación numérica (FEM)

3.15.1 5. DOFs locales deformacionales

Se reduce la descripción del estado deformacional del elemento a **3 magnitudes escalares**:

$$\mathbf{q}_l = [u_l, \bar{\theta}_1, \bar{\theta}_2]^\top, \quad u_l = l - L_0.$$

3.15.2 6. Rigidez local

En el sistema de coordenadas corrotado (eje de la barra corriente), la rigidez local 3×3 :

$$\mathbf{K}_{ll} = \begin{bmatrix} \frac{E_t A}{L_0} & 0 & 0 \\ 0 & \frac{4E_t I}{L_0} & \frac{2E_t I}{L_0} \\ 0 & \frac{2E_t I}{L_0} & \frac{4E_t I}{L_0} \end{bmatrix}.$$

Fuerzas locales $\mathbf{F}_l = \mathbf{K}_{ll} \mathbf{q}_l = [N, M_1, M_2]^\top$, con N sustituido por σA directamente del material para permitir respuestas no-lineales.

3.15.3 7. Matriz de transformación $\mathbf{B}$ (3×6)

Relación entre variaciones de $\mathbf{q}_l$ y de los DOFs globales $\delta \mathbf{u}_e$:

$$\mathbf{B} = \begin{bmatrix} -c & -s & 0 & c & s & 0 \\ -s/l & c/l & 1 & s/l & -c/l & 0 \\ -s/l & c/l & 0 & s/l & -c/l & 1 \end{bmatrix}, \quad c = \cos \alpha, \quad s = \sin \alpha.$$

c, s y l se evalúan en **configuración corriente** — esta es la esencia corrotacional.

Fuerzas internas globales:

$$\mathbf{F}_{\text{int}} = \mathbf{B}^\top \mathbf{F}_l.$$

3.15.4 8. Rigidez tangente

Se descompone en contribución material más geométrica:

$$\begin{aligned} \mathbf{K}_T &= \mathbf{K}_{\text{material}} + \mathbf{K}_\sigma, \\ \mathbf{K}_{\text{material}} &= \mathbf{B}^\top \mathbf{K}_{ll} \mathbf{B}, \\ \mathbf{K}_\sigma &= \frac{N}{l} \mathbf{z} \mathbf{z}^\top + \frac{M_1 + M_2}{l^2} (\mathbf{r} \mathbf{z}^\top + \mathbf{z} \mathbf{r}^\top), \end{aligned}$$

con vectores geométricos de 6 componentes:

$$\begin{aligned} \mathbf{r} &= [-c, -s, 0, c, s, 0]^\top \quad (\text{dirección axial}), \\ \mathbf{z} &= [-s, c, 0, s, -c, 0]^\top \quad (\text{perpendicular}). \end{aligned}$$

La parte $(N/l)\mathbf{z}\mathbf{z}^\top$ es análoga al $\mathbf{K}_G$ de **Truss2DCorot**: captura la rigidización por tensión (y el ablandamiento precrítico por compresión, base del pandeo). La parte con momentos acopla rotaciones con traslaciones — crítica para problemas de flexión con desplazamientos significativos.

3.15.5 9. Cuadratura

No aplica (forma cerrada con Hermite cúbicos).

3.16 Contrato de implementación

```

name: Frame2DEulerCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: I, type: float, required: true, desc: "Momento de inercia respecto al eje perpendicular al plano" }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar corotacional ( = (1 - L)/L)"
  nonlinearity_model: "E_tangent escala K_material local; el K_ geométrico depende solo de N, M, M corrientes"

conventions:
  sign: " > 0 > 0 (elongación); r_z positivo antihorario"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "Updated Lagrangian - l, c, s, se recalculan en cada evaluación"

validity:
  - "|_axial| 1e-2 (pequeña deformación axial)"
  - rotaciones rígidas del elemento de cualquier magnitud, pero acumuladas en |_e| < entre commits (rotaciones continuas > requerirían tracking de vueltas - fuera de alcance)
  - rotaciones deformacionales de las secciones moderadas (~|_i| 30°); para valores mayores la interpolación Hermite cúbica pierde precisión
  - vigas esbeltas (L/h 10)

out_of_scope:
  - rotaciones continuas del eje > sin pasar por commit (pendulums; aliasing en atan2)
  - deformaciones axiales grandes (requiere medida logarítmica)
  - plasticidad distribuida en la sección (fibras)
  - cortante transversal significativo (usar versión Timoshenko, pendiente)
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga a los nodos"

numerical_caveats:
  - "El ángulo _e se unwrappea a (-, ] por llamada. Si el problema tiene saltos de rotación > entre iteraciones consecutivas del solver, la formulación produce resultados incorrectos. Usar pasos de carga finos."
  - "En problemas con pandeo, la rigidez tangente se hace singular al cruzar el punto crítico. Usar `ArcLengthSolver` para seguir la rama post-crítica."

acceptance:

```



```

- name: límite lineal coincide con Frame2DEuler
  setup: "voladizo horizontal, cargas pequeñas ( ~ 1e-8, v/L ~ 1e-6)"
  expect: "K_T y F_int coinciden con Frame2DEuler lineal a tolerancia rtol=1e-4"
  tol_rel: 1.0e-4

- name: invariancia bajo rotación rígida
  setup: "viga inicialmente horizontal, rotada rígidamente 45° (ambos nodos)"
  expect: "F_int = 0, = 0 (ningún esfuerzo interno bajo rotación rígida pura)"
  tol_abs: 1.0e-10

- name: coherencia tangente/fuerza interna (chequeo por diferencias finitas)
  setup: "configuración arbitraria no trivial; K_T analítica vs derivada numérica de
        F_int"
  expect: "||K_T_analítica - K_T_numérica|| / ||K_T_analítica|| 1e-5"
  tol_rel: 1.0e-5

- name: simetría de K
  setup: "configuración arbitraria"
  expect: "K_T = K_T"
  tol_abs: 1.0e-6

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
  .1, §7.3 (corotational beams)"
- "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.11"
- "Wriggers P., Nonlinear Finite Element Methods, cap. 4"

```

3.17 Implementación

- **Archivo:** `fenix/elements/frame/euler_corot.py` — convive con `Frame2DEuler` y `Frame2DTimoshenko` en el paquete `fenix/elements/frame/`, sin heredar de ninguno. Comparte con ellos los helpers libres de `_shared.py` (carga de cuerpo, masa consistente, traducción a `ElementForces`), pero no `build_geometry_2d`: reconstruye `T` desde `alpha0` por su lógica corotacional propia.
- **Clase:** `Frame2DEulerCorot` — hereda directamente de `Element`. Métodos internos `_current_geometry` (longitud, cosenos, ángulo corriente y rigid-body α_e) y `_unwrap` (reducción a $(-\pi, \pi]$).
- **Tests:** `tests/test_frame.py · TestFrame2DEulerCorotAcceptance` — 4 criterios + registro:
- `test_acceptance_limite_lineal_coincide_con_frame2deuler`
- `test_acceptance_invariancia_rotacion_rigida`
- `test_acceptance_coherencia_tangente_fuerza_interna` — **finite-difference check** de $\mathbf{K}_T$ contra derivada numérica de $\mathbf{F}_{\text{int}}$, red de seguridad clave en esta formulación.
- `test_acceptance_simetria_K`
- `test_registro_en_registry`
- **Notas de traducción:**
- La formulación se basa en 3 DOFs deformacionales $[u, \bar{\theta}_1, \bar{\theta}_2]$ y una matriz $\mathbf{B}$ 3×6 que los relaciona con los DOFs globales. Evaluada en configuración corriente.

- **Signo de la contribución de momentos en $\mathbf{K}_\sigma$** : tras derivar $\partial(\mathbf{z}/l)/\partial\mathbf{u}_e$ analíticamente se obtiene $-(1/l^2)(\mathbf{r}\mathbf{z}^\top + \mathbf{z}\mathbf{r}^\top)$. El signo es **negativo** — un error común es ponerlo positivo. El test de diferencias finitas lo cazó en la primera implementación.
- `compute_internal_forces` proyecta $\mathbf{F}_{\text{int}}$ al sistema local corriente para reportar axial/cortante separados, utilizando la submatriz 2×2 de rotación del eje corriente.
- `_unwrap` lleva los ángulos al intervalo $(-\pi, \pi]$; la formulación es válida mientras la rotación acumulada por paso sea menor que π .

3.18 Diálogo

- **2026-04-21** · Implementación siguiendo Crisfield §7.3. Convive en `frame.py` con las vigas lineales por familia temática (todas son vigas 2D), sin herencia ni helpers compartidos.
- **2026-05-13** · `frame.py` se parte en paquete `fenix/elements/frame/`. EulerCorot vive ahora en `euler_corot.py`, sin herencia con las otras vigas 2D. Pasan a compartirse los helpers libres de carga de cuerpo, masa y traducción de fuerzas a través de `_shared.py`; la lógica corotacional propia (reconstrucción de $\mathbf{T}$ desde `alpha0`, cinemática actual) se mantiene íntegra dentro de la clase.
- **2026-04-21** · La primera versión tenía el signo equivocado en la parte de momentos de $\mathbf{K}_\sigma$ (+ en lugar de -). El test de coherencia tangente/fuerza interna por diferencias finitas lo detectó inmediatamente: $\|\mathbf{K}_{T,\text{analítica}} - \mathbf{K}_{T,\text{FD}}\|_{\text{rel}}$ del orden de $1e-1$ en lugar de $1e-5$. Tras derivar $\partial(\mathbf{z}/l)/\partial\mathbf{u}_e$ cuidadosamente (ver notas) el signo correcto es negativo. Este episodio justifica retrospectivamente incluir un test FD como criterio de aceptación en toda formulación no-lineal geométrica futura.
- **2026-04-21** · El test del límite lineal usa $P = 10$ N para obtener $v/L \sim 10^{-4}$: suficientemente pequeño para estar en régimen lineal pero suficientemente grande para que el residuo del solver converja por debajo de `tol=1e-8` (cargas menores producen desplazamientos del orden del ruido numérico y el Newton oscila en la tolerancia relativa).

3.19 Frame3D

*Orden de trabajo. El usuario escribe **especificación física, formulación numérica y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.20 Especificación física

3.20.1 0. Descripción general

Elemento 1D inmerso en el espacio tridimensional que modela una **viga esbelta 3D**. Dos nodos rígidamente conectados, 6 DOFs por nodo ($u_x, u_y, u_z, r_x, r_y, r_z$). Captura seis acciones internas:

- **Axial** (esfuerzo normal N a lo largo del eje local x).
- **Cortante en dos planos** (V_y en el plano xy , V_z en el plano xz).
- **Flexión en dos planos** (M_z alrededor del eje local z , M_y alrededor del eje local y).
- **Torsión** (M_x alrededor del eje longitudinal).

Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones). Hipótesis de Euler-Bernoulli: secciones planas perpendiculares al eje deformado (sin cizalladura transversal).

3.20.2 1. Hipótesis

- Secciones planas y perpendiculares al eje neutro deformado (igual que Frame2DEuler).
- Flexiones en los dos planos **desacopladas** entre sí (ejes principales de inercia).
- Torsión de Saint-Venant pura (sin alabeo). Válida para secciones macizas o cerradas; abiertas de pared delgada requerirían formulación con alabeo (fuera de alcance).
- Material isótropo 1D; $G = E/[2(1 + \nu)]$.

3.20.3 2. Sistema de ejes locales

Tres ejes ortonormales en cada elemento:

- **$\hat{\mathbf{x}}$ local**: dirección del eje de la barra ($\hat{\mathbf{x}} = (\mathbf{x}_2 - \mathbf{x}_1)/L$).
- **$\hat{\mathbf{y}}$ local y $\hat{\mathbf{z}}$ local**: ejes principales de inercia de la sección, ortogonales al eje.

La orientación de $\hat{\mathbf{y}}, \hat{\mathbf{z}}$ no está determinada solo por la dirección del eje — hace falta un **vector de referencia** proporcionado por el usuario (o un default). Es la diferencia estructural respecto al caso 2D, donde el plano era único.

3.20.4 3. Medidas de deformación / acciones internas

- Axial: $\varepsilon_{axial} = du/ds$, tensión $N = EA \varepsilon_{axial}$ (o $N = A \sigma$ con σ del material).
- Flexión en plano xy : curvatura $\kappa_z = d^2v/ds^2$, momento $M_z = EI_z \kappa_z$.
- Flexión en plano xz : curvatura $\kappa_y = d^2w/ds^2$, momento $M_y = EI_y \kappa_y$.
- Torsión: $d\theta_x/ds$, momento torsor $M_x = GJ d\theta_x/ds$.

3.20.5 4. Equilibrio — forma débil

PTV. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N \delta \varepsilon + M_z \delta \kappa_z + M_y \delta \kappa_y + M_x \delta \theta'_x) ds.$$

3.21 Formulación numérica (FEM)

3.21.1 5. Discretización

Dos nodos, 6 DOFs por nodo:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, r_x^{(1)}, r_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}, r_x^{(2)}, r_y^{(2)}, r_z^{(2)}]^\top.$$

Vector de 12 componentes $\rightarrow$ matrices 12×12 .

3.21.2 6. Construcción de ejes locales

Dados $\mathbf{X}_1, \mathbf{X}_2$ y un vector de referencia $\mathbf{v}_{\text{ref}}$ proporcionado:

1. $\hat{\mathbf{x}} = (\mathbf{X}_2 - \mathbf{X}_1)/L$.
2. Proyectar $\mathbf{v}_{\text{ref}}$ al plano perpendicular a $\hat{\mathbf{x}}$:

$$\tilde{\mathbf{y}} = \mathbf{v}_{\text{ref}} - (\mathbf{v}_{\text{ref}} \cdot \hat{\mathbf{x}}) \hat{\mathbf{x}}.$$

1. $\hat{\mathbf{y}} = \tilde{\mathbf{y}}/\|\tilde{\mathbf{y}}\|$ (falla si $\|\tilde{\mathbf{y}}\| \approx 0$: $\mathbf{v}_{\text{ref}}$ paralelo al eje).
2. $\hat{\mathbf{z}} = \hat{\mathbf{x}} \times \hat{\mathbf{y}}$.

Default de $\mathbf{v}_{\text{ref}}$ (cuando el usuario no lo proporciona): $[0, 0, 1]$ (eje z global como "vertical"). Si el eje del elemento es cercanamente paralelo a z global (viga vertical), se usa $[1, 0, 0]$ como fallback y se emite una advertencia.

3.21.3 7. Matriz de transformación global $\rightarrow$ local

$$\boldsymbol{\lambda} = \begin{bmatrix} \hat{\mathbf{x}}^\top \\ \hat{\mathbf{y}}^\top \\ \hat{\mathbf{z}}^\top \end{bmatrix} \quad (3 \times 3, \text{ ortogonal}).$$

$$\mathbf{T} = \text{bloques diag}(\boldsymbol{\lambda}, \boldsymbol{\lambda}, \boldsymbol{\lambda}, \boldsymbol{\lambda}) \quad (12 \times 12).$$

Aplicada a los 4 bloques de 3 DOFs: desplazamientos nodo 1, rotaciones nodo 1, desplazamientos nodo 2, rotaciones nodo 2. Las rotaciones infinitesimales en 3D transforman como vectores.

$$\mathbf{u}_{\text{local}} = \mathbf{T} \mathbf{u}_e.$$

3.21.4 8. Matriz de rigidez local

Desacoplada en cuatro contribuciones: axial, torsión, flexión xy (plano con u_y y r_z , inercia I_z), flexión xz (plano con u_z y r_y , inercia I_y).

Definiciones:

$$a_z = \frac{12EI_z}{L^3}, \quad b_z = \frac{6EI_z}{L^2}, \quad c_z = \frac{4EI_z}{L}, \quad d_z = \frac{2EI_z}{L},$$

$$a_y = \frac{12EI_y}{L^3}, \quad b_y = \frac{6EI_y}{L^2}, \quad c_y = \frac{4EI_y}{L}, \quad d_y = \frac{2EI_y}{L}.$$

La rigidez $\mathbf{K}_{\text{local}}$ es simétrica, con los bloques desacoplados (0s fuera de su plano correspondiente). Los signos del bloque de flexión xz se invierten respecto al xy porque u_z y r_y forman una pareja dextrógira opuesta (consecuencia del producto vectorial $\hat{\mathbf{x}} \times \hat{\mathbf{y}} = \hat{\mathbf{z}}$).

Forma concreta en términos de los 12 DOFs locales (la matriz completa vive en la implementación).

3.21.5 9. Fuerzas internas

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}} \mathbf{u}_{\text{local}},$$

con la componente axial corregida por el σ que devuelve el material: $F_1 = -\sigma A$, $F_7 = +\sigma A$. El resto de componentes viene del producto lineal (permite materiales no-lineales en la rama axial).

3.21.6 10. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top \mathbf{K}_{\text{local}} \mathbf{T}, \quad \mathbf{F}_{\text{int}} = \mathbf{T}^\top \mathbf{F}_{\text{int,local}}.$$

3.21.7 11. Cuadratura

No aplica (forma cerrada; integración analítica de los polinomios de Hermite).

3.22 Contrato de implementación

```
name: Frame3D
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz, rx, ry, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: Iy, type: float, required: true, desc: "Momento de inercia alrededor del eje local y" }
  - { name: Iz, type: float, required: true, desc: "Momento de inercia alrededor del eje local z" }
  - { name: J, type: float, required: true, desc: "Constante torsional de Saint-Venant" }
  - { name: nu, type: float, required: false, default: 0.3, desc: "Coeficiente de Poisson (si el material no lo expone)" }
  - { name: ref_vector, type: list, required: false, default: "[0, 0, 1]",
```

```

    desc: "Vector 3D de referencia que fija la orientación de los ejes locales y, z
          de la sección. Default: eje z global." }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar ( = (u_2 - u_1)/L en sistema local)"
  nonlinearity_model: "E_tangent escala toda la matriz local; G se recalcula como E_t
                      /[2(1+)]"
  poisson_source: "material.nu si existe; en su defecto parámetro `nu` del elemento"

conventions:
  sign: " > 0    > 0 (elongación); momentos siguen regla de la mano derecha respecto a
         los ejes locales"
  local_axes: "x_local = eje de la barra; y_local, z_local = ejes principales de
               inercia definidos por ref_vector"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "inicial fija - L, T, ejes locales no se actualizan"

validity:
  - "vigas esbeltas (L/h  10 en ambos planos de flexión)"
  - "pequeños desplazamientos y pequeñas rotaciones en el espacio"
  - "|_axial|  1e-2"
  - "sección con ejes principales de inercia bien definidos por ref_vector"

out_of_scope:
  - grandes rotaciones espaciales (requeriría Frame3DCorot; pendiente)
  - torsión con alabeo restringido (secciones abiertas de pared delgada)
  - plasticidad distribuida en la sección (fibras)
  - pandeo por bifurcación
  - deformación por cortante transversal significativa (Timoshenko 3D, fuera de alcance
    )
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga a los nodos"

numerical_caveats:
  - "Si ref_vector es paralelo al eje de la barra, la proyección al plano perpendicular
    se anula y el constructor aborta con mensaje claro - el usuario debe elegir otro
    vector."
  - "Para barras verticales (eje casi paralelo a z global) con ref_vector default, el
    constructor usa [1, 0, 0] como fallback y emite advertencia por consola."

acceptance:
  - name: respuesta axial pura
    setup: "voladizo alineado con eje x, carga F axial en extremo libre"
    expect: "u_x = F·L/(E·A); demás componentes nulas"
    tol_rel: 1.0e-10

  - name: flexión en plano xy (coherencia con Frame2D)
    setup: "voladizo alineado con eje x, carga P en dirección y, ref_vector = [0,0,1]"
    expect: "v_y = P·L³/(3·E·Iz), _z = P·L²/(2·E·Iz)"
    tol_rel: 1.0e-10

  - name: flexión en plano xz
    setup: "voladizo alineado con eje x, carga P en dirección z, ref_vector = [0,1,0]"
    expect: "v_z = P·L³/(3·E·Iy), _y = -P·L²/(2·E·Iy) (signo por mano derecha)"
    tol_rel: 1.0e-10

  - name: torsión pura
    setup: "voladizo con momento T aplicado alrededor del eje de la barra en el extremo
           libre"

```

```

expect: "_x = T·L/(G·J); demás componentes nulas"
tol_rel: 1.0e-10

- name: simetría de K
  setup: "viga en orientación arbitraria con ref_vector genérico"
  expect: "K_global = K_global"
  tol_abs: 1.0e-8

references:
- "Przemieniecki J.S., Theory of Matrix Structural Analysis, Tabla 11.3"
- "Cook R.D., Concepts and Applications of FEA, §2.8"
- "Bathe K.-J., Finite Element Procedures, cap. 5"

```

3.23 Implementación

- **Archivo:** `fenix/elements/frame3d.py` — archivo propio, sin dependencia de ningún otro elemento.
- **Clase:** `Frame3D` — hereda directamente de `Element`. Métodos privados `_build_local_frame` (longitud, matriz λ 3×3 y $\mathbf{T}$ 12×12) y `_build_local_stiffness` (matriz $\mathbf{K}_{\text{local}}$ 12×12 con bloques axial, torsión y flexión en ambos planos).
- **Tests:** `tests/test_frame3d.py · TestFrame3DAcceptance` — cubre los 5 criterios + validación de `ref_vector` paralelo al eje + registro:
 - `test_acceptance_respuesta_axial_pura`
 - `test_acceptance_flexion_xy_coincide_con_frame2d`
 - `test_acceptance_flexion_xz`
 - `test_acceptance_torsion_pura`
 - `test_acceptance_simetria_K`
 - `test_rechazo_ref_vector_paralelo`
 - `test_registro_en_registry`
- **Notas de traducción:**
 - La matriz λ 3×3 se construye como $[\hat{\mathbf{x}}^\top; \hat{\mathbf{y}}^\top; \hat{\mathbf{z}}^\top]$ — las filas son los ejes locales expresados en coordenadas globales. $\mathbf{T}$ 12×12 es un `blkdiag` de cuatro copias de λ (traslaciones y rotaciones transforman como vectores en 3D infinitesimal).
 - El default de `ref_vector` es `[0, 0, 1]`; el fallback `[1, 0, 0]` se activa cuando $|\hat{\mathbf{x}} \cdot \hat{\mathbf{z}}_{\text{global}}| > 0,99$ (barra cercanamente vertical). Ambas rutas emiten advertencia por consola la primera vez que se usan con default.
 - La matriz $\mathbf{K}_{\text{local}}$ se construye rellenando entradas no nulas en sus 4 bloques desacoplados (axial, torsión, flexión xy , flexión xz). Los signos de la flexión xz están invertidos respecto a la flexión xy porque la pareja (u_z, r_y) tiene orientación dextrógira opuesta a (u_y, r_z) : este punto se verifica explícitamente en `test_acceptance_flexion_xz`.
 - `compute_internal_forces` devuelve cortantes, torsión y momentos en ambos extremos separados — útil para post-proceso estructural (diagramas de esfuerzos).

3.24 Diálogo

- **2026-04-21** · Elemento creado como componente totalmente autónomo: no hereda de `Frame2DEuler` ni de ninguna otra viga. La maquinaria cinemática 3D se implementa íntegra dentro de `frame3d.py`, coherente con la filosofía aplicada a cables.
- **2026-04-21** · Decisión sobre `ref_vector` vs "tercer nodo de orientación". Se optó por `ref_vector` (vector 3D proyectado al plano perpendicular al eje) por ser más compacto en YAML y no requerir nodos fantasma en la malla. El default `[0, 0, 1]` cubre la mayoría de casos de ingeniería estructural (estructuras terrestres con gravedad en $-z$); el fallback para barras verticales usa `[1, 0, 0]` con advertencia explícita.
- **2026-04-21** · Verificación cruzada con `Frame2DEuler`: el criterio 2 aplica una viga alineada con el eje x global bajo carga en y ; los desplazamientos resultantes deben coincidir con la solución 2D clásica $v = PL^3/(3EI_z)$, $\theta = PL^2/(2EI_z)$. El test lo verifica a 8 decimales. Esto blindo la coherencia dimensional: el 3D se reduce al 2D sin error acumulado cuando el problema es efectivamente plano.

Capítulo 4

Elementos 2D — Sólidos

4.1 Quad4

Documentación retroactiva del elemento ya implementado. Cubre la formulación, el contrato de implementación y la trazabilidad con tests existentes.

4.2 Especificación física

4.2.1 0. Descripción general

Elemento sólido bidimensional plano de primer orden, cuadrilátero de cuatro nodos. Aproximación bilineal C^0 del campo de desplazamientos en coordenadas naturales $(\xi, \eta) \in [-1, 1]^2$. Formulación isoparamétrica: la geometría y los desplazamientos comparten las mismas funciones de forma. Régimen de pequeños desplazamientos y deformaciones.

4.2.2 1. Cinemática de desplazamientos

$$u_x(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) u_x^{(i)}, \quad u_y(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) u_y^{(i)}.$$

4.2.3 2. Cinemática de deformaciones

Tensor infinitesimal en notación Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$:

$$\varepsilon_{xx} = \frac{\partial u_x}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial u_y}{\partial y}, \quad \gamma_{xy} = \frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x}.$$

4.2.4 3. Ecuación constitutiva

Delegada al material 2D (STRAIN_DIM = 3). El elemento llama `material.compute_state( $\varepsilon$ , state)  $\rightarrow$  ( $\sigma$ , C_tangent, state')` por punto de Gauss y soporta cualquier ley constitutiva 2D (Elastic2D, IsotropicDamage2D, VonMises2D).

4.2.5 4. Equilibrio — forma fuerte

$$-\nabla \cdot \boldsymbol{\sigma} = \mathbf{b} \quad \text{en } \Omega, \quad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \quad \text{en } \partial\Omega_N, \quad \mathbf{u} = \bar{\mathbf{u}} \quad \text{en } \partial\Omega_D.$$

4.2.6 5. Equilibrio — forma débil

$$\int_{\Omega} \delta \boldsymbol{\varepsilon}^{\top} \boldsymbol{\sigma} dV = \int_{\Omega} \delta \mathbf{u}^{\top} \mathbf{b} dV + \int_{\partial\Omega_N} \delta \mathbf{u}^{\top} \bar{\mathbf{t}} dS, \quad \forall \delta \mathbf{u} \in V_0.$$

4.3 Formulación numérica (FEM)

4.3.1 6. Discretización

Cuatro nodos en orden antihorario sobre el plano (x, y) . Mapeo isoparamétrico desde el cuadrado de referencia $[-1, 1]^2$:

$$x(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) x^{(i)}, \quad y(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) y^{(i)}.$$

Jacobiano $\mathbf{J} = \partial(x, y)/\partial(\xi, \eta)$ y su determinante $\det \mathbf{J}$ entran en el cambio de variable de la integración. El código aborta con error si $\det \mathbf{J} \leq \text{tol}$ (elemento degenerado o nodos en orden invertido).

4.3.2 7. Funciones de forma

Producto tensorial bilineal:

$$N_i(\xi, \eta) = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta), \quad (\xi_i, \eta_i) = (\pm 1, \pm 1).$$

Convención de numeración (antihorario): $(\xi_1, \eta_1) = (-1, -1)$, $(\xi_2, \eta_2) = (+1, -1)$, $(\xi_3, \eta_3) = (+1, +1)$, $(\xi_4, \eta_4) = (-1, +1)$.

4.3.3 8. Matriz deformación-desplazamiento

Las derivadas en globales se obtienen vía $\partial N_i / \partial x = \mathbf{J}^{-1} \partial N_i / \partial \xi$. La matriz $\mathbf{B}$ de tamaño 3×8 se ensambla por nodo:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i / \partial x & 0 \\ 0 & \partial N_i / \partial y \\ \partial N_i / \partial y & \partial N_i / \partial x \end{bmatrix}, \quad \boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u}_e.$$

4.3.4 9. Rigidez elemental

$$\mathbf{K}_e = \int_{\Omega} \mathbf{B}^{\top} \mathbf{D} \mathbf{B} t dA = \sum_{p=1}^{n_g} \mathbf{B}(\xi_p, \eta_p)^{\top} \mathbf{D}_p \mathbf{B}(\xi_p, \eta_p) \det \mathbf{J}_p w_p t,$$

donde t es el espesor (**thickness**) y $\mathbf{D}_p$ es el tensor constitutivo tangente devuelto por el material en cada punto.

4.3.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sum_{p=1}^{n_g} \mathbf{B}(\xi_p, \eta_p)^\top \boldsymbol{\sigma}_p \det \mathbf{J}_p w_p t.$$

4.3.6 11. Cuadratura

Por defecto Gauss–Legendre 2×2 (cuatro puntos): orden de exactitud 3 — integra exactamente $\mathbf{K}_e$ para Jacobiano constante (geometría de paralelogramo) y captura el comportamiento dominante en geometrías irregulares moderadas. Soporta esquemas alternativos vía el parámetro `quadrature` (lista nombrada en `QuadratureRegistry`).

Aviso sobre integración reducida: el esquema 1×1 está disponible pero el código emite advertencia explícita; un único punto central no detecta los modos de hourglass (energía nula con desplazamientos de signo alternado), que pueden contaminar la respuesta sin estabilización adicional. No se implementa estabilización tipo Flanagan-Belytschko.

4.3.7 12. Cargas distribuidas consistentes

Fuerza de cuerpo $\mathbf{b}$ (e.g. peso propio $\mathbf{b} = (0, -\rho g)$, fuerza centrífuga). Vector nodal equivalente:

$$\mathbf{f}_e = \int_{\Omega_e} \mathbf{N}^\top \mathbf{b} t dA = \sum_{p=1}^{n_g} \mathbf{N}(\xi_p, \eta_p)^\top \mathbf{b} \det \mathbf{J}_p w_p t,$$

donde $\mathbf{N}$ es la matriz 2×8 de funciones de forma. Se reutiliza la cuadratura del elemento (Gauss 2×2 por defecto) — exacta para $\mathbf{b}$ uniforme y geometrías de paralelogramo, y adecuada para geometrías irregulares moderadas.

Tracción de superficie $\bar{\mathbf{t}}$ sobre un borde $\Gamma_e \subset \partial\Omega_N$:

$$\mathbf{f}_e = \int_{\Gamma_e} \mathbf{N}^\top \bar{\mathbf{t}} t dS.$$

Cada borde es una recta entre dos nodos; sobre él las funciones de forma son lineales y, para $\bar{\mathbf{t}}$ constante, la integral se reduce a un reparto exacto de $\frac{L}{2} \bar{\mathbf{t}} t$ a cada uno de los dos nodos del borde, donde L es la longitud del segmento. Bordes numerados según la conectividad nodal:

Edge	Nodos
0	(0, 1)
1	(1, 2)
2	(2, 3)
3	(3, 0)

$\bar{\mathbf{t}}$ se especifica en **coordenadas globales** (t_x, t_y) ; presiones normales se obtienen multiplicando previamente por la normal exterior del borde. Tracción variable a lo largo del borde no está soportada en este paso.

4.3.8 13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve un dict con $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ en cada uno de los n_g puntos de Gauss del elemento, junto con sus coordenadas naturales y globales. Habilita post-proceso fino (mapas no promediados, suavizado nodal, extrapolación tipo Barlow). `compute_internal_forces` queda como atajo de promedio.

4.4 Contrato de implementación

```

name: Quad4
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 4
  strain_dim: 3          # Voigt 2D [_xx, _yy, _xy]
  n_integration_points: 4 # default Gauss 2x2

parameters:
  - { name: thickness, type: float, required: false, default: 1.0, desc: Espesor para
    estado plano }
  - { name: quadrature, type: tuple, required: false, default: "2x2", desc: Regla de
    cuadratura desde QuadratureRegistry }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state)"
  strain_kind: "Voigt 2D [_xx, _yy, _xy]"

conventions:
  sign: "tensión positiva = tracción"
  voigt: "[xx, yy, xy] con _xy = 2·_xy (engineering shear strain)"
  node_orientation: "antihorario (asegura det J > 0)"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "geometría sin distorsión severa (det J > tol en todos los puntos de Gauss)"
  - "compatible con materiales Elastic2D, IsotropicDamage2D, VonMises2D (este último
    solo plane_strain)"

out_of_scope:
  - "grandes deformaciones, formulaciones corrotacionales o totales lagrangianas"
  - "estabilización contra hourglass para integración reducida"
  - "elementos de alto orden (Q8, Q9)"

acceptance:
  verification:
    - name: patch_test_macneal_harder
      setup: "5 Quad4 distorsionados (4 nodos exteriores + 4 interiores)
        con campo lineal u = 1e-3·(x + y/2), v = 1e-3·(y + x/2)
        impuesto en los 4 nodos del contorno"
      expect: "nodos interiores adoptan el campo lineal y = (1e-3, 1e-3,
        1e-3) en todos los puntos de Gauss; uniforme"
      tol_rel: 1.0e-10
  specific:
    - name: parche de tracción uniaxial sobre malla regular
      setup: "malla regular con campo de desplazamiento lineal y material Elastic2D"
      expect: "_xx constante en todos los puntos de Gauss"
      tol_rel: 1.0e-12
    - name: jacobiano degenerado abortado
      setup: "elemento con nodos colineales o en orden inverso"
      expect: "ValueError en _compute_kinematics"
    - name: cargas consistentes - body load uniforme
      setup: "Quad4 regular y distorsionado con b uniforme; sumar componentes nodales"
      expect: "Σf_x = b_x·A·t y Σf_y = b_y·A·t (invariante de geometría)"

```

```

tol_rel: 1.0e-10
- name: cargas consistentes - tracción uniforme en un borde
  setup: "Quad4 con tracción constante en un borde"
  expect: "Σf = -t·L·t y reparto L/2 a cada nodo del borde, 0 en los demás"
  tol_rel: 1.0e-12
- name: patch físico de tracción uniaxial
  setup: "Quad4 cuadrado L×H, borde izquierdo con ux=0 (n0,n3) + uy=0 (n0); borde
    derecho con tracción uniforme (p,0)"
  expect: "u reproduce ux=p·x/E, uy=-p·y/E (plane stress); _xx=p, _yy=_xy=0"
  tol_rel: 1.0e-10

references:
- "Bathe K.-J., Finite Element Procedures, §5.3 (isoparametric elements)"
- "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §6 (plane stress/
  strain)"

```

4.5 Implementación

- **Archivo:** fenix/elements/solid_2d/quad4.py
- **Clase:** Quad4 (registrada vía @ElementRegistry.register)
- **Funciones núcleo:** `_compute_kinematics(xi, eta, coords)` y `_compute_integrands(B, C,  $\sigma$ , detJ, w, t)`, ambas decoradas con @njit (Numba) para evaluar **B**, det **J** y los integrandos a velocidad cercana a Fortran. Viven en fenix/elements/solid_2d/_shared.py y se comparten con Tri3.
- **Tests:**
- tests/test_solid_2d.py — ensamblaje de **B**, simetría de **K_e**, modos de cuerpo rígido y tracción uniaxial sobre malla regular.
- tests/test_patch_solid_2d.py — patch test de MacNeal-Harder sobre 5 Quad4 distorsionados (NAFEMS, V&V).

4.6 Diálogo

- **2026-04-30** · Spec retroactiva. Quad4 precedía al protocolo spec-first; la spec se creó documentando la formulación ya implementada y verificada. Promovido **status: validated**.
- **2026-04-30** · Añadido patch test de MacNeal-Harder (NAFEMS) en **acceptance verification**. Cinco Quad4 distorsionados con cuatro nodos interiores libres reproducen exactamente un campo lineal impuesto en el contorno, con ε constante e igual al gradiente analítico en todos los puntos de Gauss.
- **2026-05-04** · Expuesta `compute_gauss_state(U)` con σ y ε por punto de Gauss y coordenadas naturales/globales. `compute_internal_forces` queda como promedio derivado. El `VtkExporter` añade campos nodales `Sigma_XX_nodal`, `Sigma_YY_nodal`, `Tau_XY_nodal` y `Von_Mises_nodal` por promedio simple sobre los elementos contiguos a cada nodo (suavizado de orden 0).

- **2026-05-04** · Añadidas cargas distribuidas consistentes (`compute_body_load`, `compute_edge_traction`). Para tracciones uniformes en bordes rectos el reparto es exacto ($L/2$ a cada nodo del borde) y se evita la cuadratura 1D; las fuerzas de cuerpo se integran con la cuadratura del elemento. Solo tracciones **constantes por borde** y en **coordenadas globales** en este paso; tracción variable y presión normal quedan para una iteración posterior.

4.7 Tri3

Documentación retroactiva del elemento ya implementado.

4.8 Especificación física

4.8.1 0. Descripción general

Elemento sólido bidimensional plano de primer orden, triangular de tres nodos. Aproximación lineal C^0 del campo de desplazamientos en coordenadas naturales (ξ, η) . Conocido en la literatura como elemento *constant strain triangle* (CST): la matriz $\mathbf{B}$ es constante sobre el elemento, por lo que la deformación $\boldsymbol{\varepsilon}$ y la tensión $\boldsymbol{\sigma}$ son uniformes dentro de cada Tri3. Régimen de pequeños desplazamientos y deformaciones.

4.8.2 1. Cinemática de desplazamientos

$$u_x(\xi, \eta) = \sum_{i=1}^3 N_i(\xi, \eta) u_x^{(i)}, \quad u_y(\xi, \eta) = \sum_{i=1}^3 N_i(\xi, \eta) u_y^{(i)}.$$

4.8.3 2. Cinemática de deformaciones

Notación Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$, idéntica a la de Quad4. Por la linealidad de N_i , las derivadas $\partial N_i / \partial x$ son constantes y $\boldsymbol{\varepsilon}$ es uniforme en el elemento.

4.8.4 3. Ecuación constitutiva

Delegada al material 2D (STRAIN_DIM = 3). Soporta Elastic2D, IsotropicDamage2D, VonMises2D (este último solo plane_strain).

4.8.5 4. Equilibrio — forma fuerte

$$-\nabla \cdot \boldsymbol{\sigma} = \mathbf{b} \quad \text{en } \Omega, \quad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \quad \text{en } \partial\Omega_N, \quad \mathbf{u} = \bar{\mathbf{u}} \quad \text{en } \partial\Omega_D.$$

4.8.6 5. Equilibrio — forma débil

Idéntica a la del Quad4; el principio variacional no depende de la forma del elemento.

4.9 Formulación numérica (FEM)

4.9.1 6. Discretización

Tres nodos en orden antihorario sobre el plano (x, y) . Mapeo isoparamétrico desde el triángulo de referencia con vértices en $(0, 0)$, $(1, 0)$, $(0, 1)$. Jacobiano $\mathbf{J} = \partial(x, y) / \partial(\xi, \eta)$ es constante sobre el elemento; el código aborta con error si $\det \mathbf{J} \leq \text{tol}$ (nodos colineales o invertidos).

4.9.2 7. Funciones de forma

Coordenadas baricéntricas:

$$N_1 = 1 - \xi - \eta, \quad N_2 = \xi, \quad N_3 = \eta.$$

Sus derivadas en coordenadas naturales son constantes:

$$\begin{aligned} \partial N_1 / \partial \xi &= -1, & \partial N_2 / \partial \xi &= 1, & \partial N_3 / \partial \xi &= 0; \\ \partial N_1 / \partial \eta &= -1, & \partial N_2 / \partial \eta &= 0, & \partial N_3 / \partial \eta &= 1. \end{aligned}$$

4.9.3 8. Matriz deformación-desplazamiento

$\mathbf{B}$ tiene tamaño 3×6 y, una vez transformadas las derivadas a globales vía $\mathbf{J}^{-1}$, sus entradas son **constantes** sobre el elemento:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i / \partial x & 0 \\ 0 & \partial N_i / \partial y \\ \partial N_i / \partial y & \partial N_i / \partial x \end{bmatrix}.$$

4.9.4 9. Rigidez elemental

$$\mathbf{K}_e = \mathbf{B}^\top \mathbf{D} \mathbf{B} A_e t,$$

con $A_e = \frac{1}{2} \det \mathbf{J}$ el área del triángulo y t el espesor. La integración es exacta con un único punto central porque $\mathbf{B}$ es constante.

4.9.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \mathbf{B}^\top \boldsymbol{\sigma} A_e t.$$

4.9.6 11. Cuadratura

Un punto central (en el baricentro) con peso $w = 1/2$ sobre el triángulo de referencia. Es exacto para formas constantes de $\mathbf{B}$ y $\boldsymbol{\sigma}$.

4.9.7 12. Cargas distribuidas consistentes

Fuerza de cuerpo $\mathbf{b}$. Con N_i lineales y $\mathbf{b}$ uniforme la integral es exacta:

$$\mathbf{f}_e^b = \int_{\Omega_e} \mathbf{N}^\top \mathbf{b} t dA \implies \text{cada nodo recibe } \frac{1}{3} \mathbf{b} A_e t.$$

Tracción de superficie $\bar{\mathbf{t}}$ sobre un borde recto. Con $\bar{\mathbf{t}}$ constante el reparto es $\frac{L}{2} \bar{\mathbf{t}} t$ en cada uno de los dos nodos del borde. Bordes numerados según la conectividad:

Edge	Nodos
0	(0, 1)
1	(1, 2)
2	(2, 0)

$\bar{\mathbf{t}}$ se especifica en coordenadas globales (t_x, t_y) . Tracción variable o presión normal no soportadas en este paso.

4.9.8 13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve la misma estructura que en `Quad4` pero con un único punto (centroide). Para `Tri3` ε y σ son constantes sobre el elemento, así que el dato del punto coincide con el promedio.

4.10 Limitación crítica — *shear locking*

El `Tri3` es notoriamente rígido en flexión: tres DOFs de desplazamiento no bastan para representar el modo de flexión puro de una viga discretizada en triángulos, por lo que el elemento responde con cortante espurio (*shear locking*) y subestima sistemáticamente la deflexión. La severidad crece con la relación L/h del problema. Recomendación operativa: usar `Quad4` por defecto; reservar `Tri3` para zonas de transición de malla en regiones donde el campo de tensiones varía suavemente.

4.11 Contrato de implementación

```
name: Tri3
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 3
  strain_dim: 3          # Voigt 2D [_xx, _yy, _xy]
  n_integration_points: 1

parameters:
  - { name: thickness, type: float, required: false, default: 1.0, desc: Espesor para
      estado plano }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state)"
  strain_kind: "Voigt 2D [_xx, _yy, _xy]"

conventions:
  sign: "tensión positiva = tracción"
  voigt: "[xx, yy, xy] con _xy = 2·_xy"
  node_orientation: "antihorario (asegura det J > 0)"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "campos de tensión que varían suavemente sobre el elemento"

out_of_scope:
  - "flexión dominante (shear locking severo)"
  - "elementos de alto orden (Tri6)"
  - "grandes deformaciones"

acceptance:
  verification:
    - name: patch_test_macneal_harder
      setup: "4 Tri3 distorsionados alrededor de un nodo interior descentrado
              con campo lineal u = 1e-3·(x + y/2), v = 1e-3·(y + x/2)
              impuesto en los 4 nodos del contorno del cuadrado unitario"
      expect: "nodo interior adopta el campo lineal y = (1e-3, 1e-3,"
```

```

        1e-3) en cada elemento;  uniforme"
    tol_rel: 1.0e-10
specific:
- name: parche de deformación constante sobre malla triangular regular
  setup: "malla triangular con campo de desplazamiento lineal y material Elastic2D"
  expect: " constante (CST reproduce campos lineales por construcción)"
  tol_rel: 1.0e-12
- name: jacobiano degenerado abortado
  setup: "tres nodos colineales"
  expect: "ValueError en _compute_kinematics_tri3"
- name: cargas consistentes - body load uniforme
  setup: "Tri3 con b uniforme"
  expect: "cada nodo recibe (1/3)·b·A_e·t;  $\Sigma = b \cdot A_e \cdot t$ "
  tol_rel: 1.0e-12
- name: cargas consistentes - tracción uniforme en un borde
  setup: "Tri3 con tracción constante en un borde"
  expect: " $\Sigma_f = t \cdot L \cdot t$  y reparto L/2 a cada nodo del borde, 0 en el opuesto"
  tol_rel: 1.0e-12

references:
- "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §3.6 (CST)"
- "Zienkiewicz O.C., The Finite Element Method, vol. 1, §5 (limitations of CST)"

```

4.12 Implementación

- **Archivo:** `fenix/elements/solid_2d/tri3.py`
- **Clase:** `Tri3` (registrada vía `@ElementRegistry.register`)
- **Función núcleo:** `_compute_kinematics_tri3(coords)`, decorada con `@njit`, en `_shared.py`. Reutiliza `_compute_integrands` (también en `_shared`, compartido con `Quad4`) con peso $w = 0.5$.
- **Tests:**
- `tests/test_solid_2d.py` — patch test sobre malla triangular regular y modos de cuerpo rígido.
- `tests/test_patch_solid_2d.py` — patch test de MacNeal-Harder con nodo interior libre (NAFEMS, V&V).

4.13 Diálogo

- **2026-04-30** · Spec retroactiva. `Tri3` precedía al protocolo `spec-first`; la spec se creó documentando la formulación ya implementada. Promovido `status: validated`.
- **2026-04-30** · Añadido patch test de MacNeal-Harder (NAFEMS) en `acceptance.verificaton`. Cuatro triángulos alrededor de un nodo interior descentrado reproducen un campo lineal impuesto en el contorno con ε constante en cada elemento.
- **2026-05-04** · Expuesta `compute_gauss_state(U)` (1 punto central). El `VtkExporter` consume σ por elemento y promedia a nodos (`Sigma_*_nodal`).

- **2026-05-04** · Añadidas cargas distribuidas consistentes (`compute_body_load`, `compute_edge_traction`) con la misma semántica que `Quad4`. Para `Tri3` ambos integrandos son exactos analíticamente: body load reparte $1/3$ por nodo, tracción de borde reparte $L/2$ a cada uno de los dos nodos del borde.

Capítulo 5

Modelos Constitutivos (Materiales)

5.1 CableMaterial1D

*Orden de trabajo. El usuario escribe **especificación física, formulación y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

5.2 Especificación física

5.2.1 0. Descripción general

Material 1D memoryless que modela la respuesta axial de un cable: **elástico lineal en tensión y nulo en compresión**. No tiene variables internas ni historia — la respuesta depende exclusivamente de la deformación instantánea. Captura la propiedad esencial de un cable: que solo transmite esfuerzo cuando está tensado.

5.2.2 1. Ecuación constitutiva

$$\sigma(\varepsilon) = E \langle \varepsilon \rangle^+ = \begin{cases} E \varepsilon & \varepsilon > 0 \\ 0 & \varepsilon \leq 0 \end{cases}$$

Rampa lineal de pendiente E en tensión ($\varepsilon > 0$), cero en compresión o estado sin deformación ($\varepsilon \leq 0$). $\langle \cdot \rangle^+$ denota la parte positiva (operador de MacAuley).

5.2.3 2. Módulo tangente

$$E_t(\varepsilon) = \frac{d\sigma}{d\varepsilon} = \begin{cases} E & \varepsilon > 0 \\ 0 & \varepsilon \leq 0 \end{cases}$$

Discontinuo en $\varepsilon = 0$. Por convención el valor en el punto de corte se asigna al régimen compresivo ($E_t = 0$), lo que evita generar rigidez espuria cuando el cable está exactamente sin tensión.

5.2.4 3. Variables internas

No hay. La respuesta es puramente función de ε actual; el estado del material es el vacío.

5.2.5 4. Interpretación física

- $\varepsilon > 0$: cable tensado — se comporta como barra elástica.
- $\varepsilon < 0$: cable destensado — no transmite nada, la carga pasa por otros caminos del modelo.
- $\varepsilon = 0$: frontera neutra — no hay tensión, no hay rigidez.

La no linealidad es **constitutiva** (en la respuesta del material), no geométrica. Un elemento con este material puede tener cualquier cinemática (lineal o corotacional); la unilateralidad solo vive aquí.

5.3 Contrato de implementación

```

name: CableMaterial1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: null      # memoryless, no exporta variable principal

parameters:
  - { name: E, type: float, required: true, desc: "Módulo de Young (en tensión)" }

signature:
  compute_state: "(: float, state_vars=None) -> (: float, E_tangent: float, state_vars)"
  strain_kind: axial escalar

conventions:
  sign: " > 0      > 0 (elongación);      0      = 0"
  state_passthrough: "state_vars se devuelve intacto (el material no almacena historia)"

validity:
  - "E > 0"
  - "|| 1e-2 en el régimen tensado (elasticidad lineal)"
  - "IS_UNILATERAL = True solo admitido en elementos con ACCEPTS_UNILATERAL = True (Cable2DCorot, Cable3DCorot). La base Element rechaza la combinación con Truss* en construcción."

out_of_scope:
  - plasticidad, fluencia, fatiga
  - viscoelasticidad / dependencia temporal
  - pretensión inicial (el usuario la modela con una longitud de referencia ajustada)
  - regularización numérica en compresión (no hay rigidez residual en 0)
  - anisotropía direccional; la respuesta es puramente axial escalar

numerical_caveats:
  - "En transiciones tensado destensado (cruza 0), E_tangent salta de E a 0 y Newton-Raphson puede oscilar. Mitigación: usar arc-length o pasos de carga finos si el problema tiene destensiones reales."
  - "Un cable completamente destensado tiene K_M = 0 y K_G = 0 matriz tangente singular en los DOFs del elemento. El usuario debe garantizar que otros elementos del modelo proporcionen rigidez suficiente, o pretensar el cable antes de aplicar cargas que puedan destensarlo."

```

```

acceptance:
- name: respuesta en tensión pura
  setup: " = 1e-4"
  expect: " = E · 1e-4, E_tangent = E"
  tol_rel: 1.0e-12

- name: respuesta en compresión pura
  setup: " = -1e-4"
  expect: " = 0, E_tangent = 0"
  tol_abs: 1.0e-15

- name: punto de corte
  setup: " = 0"
  expect: " = 0, E_tangent = 0 (régimen compresivo por convención)"
  tol_abs: 1.0e-15

- name: state_vars se devuelve intacto
  setup: "state_vars = {'dummy': 42}, arbitrario"
  expect: "el tercer retorno es idéntico al state_vars de entrada"

references:
- "Irvine H.M., Cable Structures, MIT Press, §1.2 (modelo elástico del cable)"
- "MacAuley M.A., on the deflection of beams, 1919 (operador parte positiva)"

```

5.4 Implementación

- **Archivo:** `fenix/materials/cable_1d.py` — archivo propio, sin dependencia de otros módulos de material salvo la clase base `Material`.
- **Clase:** `CableMaterial1D` — hereda directamente de `Material` (no de `Elastic1D` ni de ningún otro material), registrada vía `@MaterialRegistry.register`.
- **Tests:** `tests/test_cable_material.py · TestCableMaterial1D` — seis tests:
 - `test_acceptance_tension_pura` (criterio 1).
 - `test_acceptance_compresion_pura` (criterio 2).
 - `test_acceptance_punto_de_corte` (criterio 3).
 - `test_acceptance_state_vars_passthrough` (criterio 4).
 - `test_registro_en_registry` — verifica autodiscover.
 - `test_validacion_E_positivo` — rechaza $E \leq 0$.
- **Notas de traducción:**
 - La ramificación en `compute_state` es un único `if strain > 0.0`. El `>` estricto implementa la convención del punto de corte (el cero se asigna al régimen compresivo). El `else` cubre tanto $\varepsilon < 0$ como $\varepsilon = 0$ con las mismas salidas.
 - `state_vars` se devuelve como llega (identidad, no copia). El test `test_acceptance_state_vars_passthrough` usa `assertIs` para confirmar identidad.

- Validación de $E > 0$ al construir, con mensaje claro. Atrapa typos del input YAML temprano.
- El material acepta ****kwargs** en `compute_state` por compatibilidad con elementos que pudieran pasar argumentos adicionales (convención del proyecto).

5.5 Diálogo

- **2026-04-21** · Material creado como pieza independiente para la futura implementación de elementos de cable (`Cable2DCorot`, `Cable3DCorot`). Deliberadamente **no hereda** de `Elastic1D` aunque su rama en tensión sea idéntica: mantener el material autocontenido simplifica su lectura y evita acoplamientos futuros si la rama elástica cambia en otro archivo.
- **2026-04-21** · Discontinuidad en $\varepsilon = 0$: documentada explícitamente en `out_of_scope: regularización numérica`. Si aparece un caso de uso con destensiones frecuentes donde Newton-Raphson oscile, se añadirá una variante `CableMaterial1DRegularized` con $E_c \ll E$ en compresión — **no** se modificará este material (contrato limpio).
- **2026-05-12** · Restricción de emparejamiento elevada a contrato: `IS_UNILATERAL = True` en el material; `ACCEPTS_UNILATERAL = True` en `Cable2DCorot/Cable3DCorot`. La base `Element._validate_material` rechaza la combinación con `Truss*` (cuyo tangente colapsa la rigidez global sin `K_G` que lo compense). Antes el contrato lo permitía y solo se desaconsejaba en docstring; ahora falla limpio al construir.

Capítulo 6

Modelos constitutivos — catálogo

Referencia rápida de los modelos constitutivos implementados. Una entrada por material. Para detalles del algoritmo → código fuente.

>**Convenciones:** `STRAIN_DIM` = dimensión Voigt de la deformación de entrada (1 = escalar, 3 = 2D [`εxx`, `εyy`, `γxy`], 6 = 3D). `PRIMARY_STATE_VAR` = variable interna que el `VtkExporter` lleva al output.

>**Densidad (ADR 0008):** todos los materiales aceptan un parámetro opcional *density* (kg/m³ en las unidades del problema). No requerido al construir — análisis estáticos sin peso propio funcionan sin declararla. Cuando un consumidor que requiere masa la pide (`Assembler.assemble_self_weight(g)` y futuras `assemble_mass_matrix`, etc.) y la encuentra como `None`, **falla con `ValueError`** listando los materiales sin densidad. Valor `0.0` declarado explícitamente cubre el caso legítimo de material sin masa física (penalty, restricción) y emite warning informativo.

6.1 Elastic1D — elástico lineal 1D

- **Ley:** $\sigma = E \cdot \varepsilon$ (Hooke 1D).
- **STRAIN_DIM:** 1.
- **Parámetros:** E (módulo de Young).
- **Variables internas:** ninguna (sin historia).
- **Tangente:** constante = E.
- **Compatible con:** Truss2D, Truss3D, Frame2DEuler, Frame2DTimoshenko.
- **Archivo:** fenix/materials/elastic.py

6.2 Elastic2D — elástico lineal 2D

- **Ley:** $\sigma = C \cdot \varepsilon$, con C el tensor constitutivo isótropo en notación Voigt [`xx`, `yy`, `xy`].
- **STRAIN_DIM:** 3.
- **Parámetros:** E, nu, hypothesis ∈ {'plane_stress', 'plane_strain'}.
- **Variables internas:** ninguna.

- **Tangente:** constante = C .
- **Compatible con:** Quad4, Tri3.
- **Archivo:** fenix/materials/elastic_2d.py

6.3 IsotropicDamage1D — daño isótropo 1D con softening exponencial

- **Modelo:** daño escalar $d \in [0, 1)$, módulo secante $E_{\text{sec}} = (1 - d) \cdot E$, $\sigma = E_{\text{sec}} \cdot \varepsilon$.
- **Evolución del daño** (Kuhn-Tucker):
- Deformación equivalente: $\varepsilon_{\text{eq}} = |\varepsilon|$.
- Variable histórica: $\kappa = \max(\kappa_{\text{old}}, \varepsilon_{\text{eq}})$.
- Si $\kappa \leq \kappa_0 \rightarrow d = 0$. Si no: $d = 1 - (\kappa_0/\kappa) \cdot \exp(-\alpha \cdot (\kappa - \kappa_0))$, saturado en $DAMAGE_MAX$.
- **STRAIN_DIM:** 1 · **PRIMARY_STATE_VAR:** 'damage'.
- **Parámetros:** E , κ_0 (umbral elástico), α (velocidad de degradación).
- **Variables internas:** κ , damage .
- **Tangente:** secante (E_{sec}); no consistente con tangente algorítmica del daño — adecuado para Newton-Raphson amortiguado, no óptimo para arc-length agresivo.
- **Admisibilidad (ADR 0006):** $\text{admissibility_scale} = E \cdot \kappa_0$ — esfuerzo umbral inicial donde se activa el daño. Garantiza invariancia bajo cambio de unidades del check $f \leq \text{atol} + \text{rtol} \cdot \text{escala}$.
- **Compatible con:** Truss2D, Truss3D.
- **Referencia:** Lemaitre & Chaboche, *Mechanics of Solid Materials*; Oliver, ^A "consistent characteristic length for smeared cracking models" (IJNME 1989).
- **Archivo:** fenix/materials/damage_1d.py

6.4 IsotropicDamage2D — daño isótropo 2D con softening exponencial

- **Modelo:** extensión 2D de IsotropicDamage1D. Tensor constitutivo secante $C_{\text{sec}} = (1 - d) \cdot C_e$.
- **Deformación equivalente:** $\varepsilon_{\text{eq}} = \sqrt{(\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \frac{1}{2} \gamma_{xy}^2)}$ (norma simple sin distinguir tensión/compresión).
- **Evolución:** idéntica a 1D (κ máxima histórica + ley exponencial).
- **STRAIN_DIM:** 3 · **PRIMARY_STATE_VAR:** 'damage'.
- **Parámetros:** E , ν , κ_0 , α , hypothesis (delegado a Elastic2D interno).

- **Variables internas:** kappa, damage.
- **Tangente:** secante.
- **Admisibilidad (ADR 0006):** `admissibility_scale` = $E \cdot \kappa_0$ — idéntica al caso 1D, el criterio de daño tiene el mismo umbral característico independientemente de la dimensión.
- **Limitación:** la ε_{eq} simétrica no distingue daño en tensión vs compresión; para hormigón usar modelo de Mazars o split tensión/compresión (no implementado).
- **Compatible con:** Quad4, Tri3.
- **Archivo:** `fenix/materials/damage_2d.py`

6.5 Elastoplastic1D — plasticidad J2 1D con endurecimiento isótropo lineal

- **Modelo:** plasticidad asociativa con criterio $f = |\sigma_{trial}| - (\sigma_y + H \cdot \alpha) \leq 0$.
- **Algoritmo:** return mapping clásico 1D.
- **Predictor elástico:** $\sigma_{trial} = E \cdot (\varepsilon - \varepsilon_{p_old})$.
- **Corrector:** $\Delta\gamma = f / (E + H)$, $\sigma = \sigma_{trial} - \Delta\gamma \cdot E \cdot \text{sign}(\sigma_{trial})$.
- **Tangente algorítmica consistente:** $E_t = E \cdot H / (E + H)$.
- **STRAIN_DIM:** 1 · **PRIMARY_STATE_VAR:** 'alpha'.
- **Parámetros:** E, `sigma_y` (fluencia inicial), H (módulo de endurecimiento; 0 = perfectamente plástico).
- **Variables internas:** `eps_p` (deformación plástica), `alpha` (acumulada equivalente).
- **Admisibilidad (ADR 0006):** `admissibility_scale` = $\sigma_y + H \cdot \alpha$ — fluencia corriente (adaptativa al endurecimiento). El check $f \leq \text{atol} + \text{rtol} \cdot \text{escala}$ se hace contra la superficie de fluencia *en el estado de entrada del paso*.
- **Compatible con:** Truss2D, Truss3D, Frame2DEuler, Frame2DTimoshenko (en estos últimos solo se aplica al esfuerzo axial).
- **Referencia:** Simo & Hughes, *Computational Inelasticity*, cap. 1.
- **Archivo:** `fenix/materials/plastic_1d.py`

6.6 CableMaterial1D — cable axial 1D con elasticidad unilateral

- **Ley:** $\sigma = E \cdot \varepsilon$ si $\varepsilon > 0$; $\sigma = 0$ si $\varepsilon \leq 0$ (sin rigidez en compresión).
- **STRAIN_DIM:** 1.
- **Parámetros:** E (módulo de Young en tensión, estrictamente positivo).

- **Variables internas:** ninguna (respuesta memoryless: la rigidez depende solo del signo instantáneo de ε).
- **Tangente:** $E_t = E$ en tensión, $E_t = 0$ en compresión o sin tensión.
- **Implicación numérica:** la rigidez tangente colapsa a cero cuando el cable se afloja, lo que requiere preconditionamiento o regularización en el solver para no degenerar la matriz global. El uso típico es a través del elemento `Cable2DCorot/Cable3DCorot`, que detecta la situación y la maneja correctamente.
- **Compatible con:** `Cable2DCorot`, `Cable3DCorot`.
- **Referencia:** ver `docs/specs/CableMaterial1D.md`.
- **Archivo:** `fenix/materials/cable_1d.py`

6.7 VonMises2D — plasticidad J2 2D con endurecimiento isótropo lineal

- **Modelo:** J2 (Von Mises) en deformación plana; criterio $f = \|s_{\text{trial}}\| - \sqrt{(2/3) \cdot (\sigma_y + H \cdot \alpha)} \leq 0$.
- **Algoritmo:** return mapping radial sobre la parte desviadora.
- Descomposición volumétrica/desviadora: $p = K \cdot \text{tr}(\varepsilon)$, $s = 2G \cdot e_{\text{dev}}$.
- Predictor elástico desviador, corrector radial $\Delta\gamma = f_{\text{trial}} / (2G + \cdot H)$.
- Actualización: $s_{\text{new}} = s_{\text{trial}} - 2G \cdot \Delta\gamma \cdot N$, con $N = s_{\text{trial}} / \|s_{\text{trial}}\|$.
- Tangente algorítmica consistente derivada por linealización del return mapping.
- **STRAIN_DIM:** 3 · **PRIMARY_STATE_VAR:** 'alpha'.
- **Parámetros:** E , ν , σ_y , H , hypothesis (obligatorio 'plane_strain').
- **Variables internas:** `eps_p` (tensorial 4 componentes [xx, yy, zz, xy]), `alpha` (acumulada equivalente).
- **Admisibilidad (ADR 0006):** $\text{admissibility_scale} = \sqrt{(2/3)} \cdot (\sigma_y + H \cdot \alpha)$ — radio en espacio desviador de la superficie de Von Mises en el estado corriente. Es la escala natural de $\|s_{\text{trial}}\|$ en la frontera de fluencia J2. Por estar el return mapping en un kernel `@njit`, la tolerancia se precomputa fuera del kernel vía `self.admissibility_tol(state_vars)` y se pasa como `float` al JIT.
- **Restricción crítica:** solo deformación plana. Usar con `hypothesis='plane_stress'` lanza `NotImplementedError` (el return mapping para plane stress requiere algoritmo diferente).
- **Implementación:** kernel `_compute_j2_plasticity` con `@njit`.
- **Compatible con:** `Quad4`, `Tri3` (configurados en plane strain).
- **Referencia:** Simo & Hughes, *Computational Inelasticity*, cap. 3 (return mapping J2).
- **Archivo:** `fenix/materials/von_mises_2d.py`

6.8 Cómo añadir un material nuevo

`/fenix-new material <Name>` — genera archivo en `fenix/materials/`, decorador `@MaterialRegistry.register` esqueleto de test. Declarar **STRAIN_DIM** y, si tiene historia, **PRIMARY_STATE_VAR** (la variable que aparecerá en el VTK). Tras implementar el modelo constitutivo, **añadir una entrada a este catálogo** siguiendo el formato de arriba.

Capítulo 7

Anexos técnicos

*Esta sección es referencia técnica del subsistema interno que resuelve el sistema lineal $K\mathbf{x} = \mathbf{b}$ que aparece en el corazón de cada solver no lineal. La elección del backend es **automática**; el usuario no decide. Esta sección documenta cómo se decide y cuáles son los puntos de override para diagnóstico.*

>Diseño completo: ver ADR 0003 en `docs/adr/0003-despachador-de-solver-algebraico.md`.

7.1 Dos capas que se llaman ambas «solver»

Conviene distinguirlas:

- **Solver no lineal** — `LinearSolver`, `NonlinearSolver`, `ArcLengthSolver`. Orquesta la estrategia de paso, las iteraciones de Newton, el control de longitud de arco, los criterios de convergencia. Es lo que el usuario elige en el YAML con `solver.type`.
- **Capa algebraica** — `fenix.math.linalg`. Resuelve el sistema lineal $K \cdot \delta\mathbf{U} = \mathbf{R}$ que aparece dentro de cada iteración del solver no lineal. Tiene varios *backends* numéricos y un despachador interno que elige el adecuado según las propiedades de K . Es plumbing automático: el usuario no la ve salvo que pida diagnóstico.

7.2 Backends disponibles y criterio de selección

El despachador (`fenix.math.linalg.dispatcher.select_solver`) elige el backend en función de tres flags declarativos sobre K : simetría, positividad y talla.

Régimen de K	Backend elegido	Notas
Simétrica + positiva definida	Cholesky (CHOLMOD)	2× más rápido y 2× menos memoria que LU. Requiere la dependencia opcional <code>scikit-sparse</code> . Si falta, degrada a LU con un <i>warning</i> una sola vez.
Simétrica indefinida	LU general (placeholder LDL^T)	Ideal para LDL^T con conteo de pivotes negativos (Sturm sequence) en pandeo y snap-through. La interfaz está reservada; mientras no haya backend nativo, se usa LU sin pérdida de corrección, solo sin diagnóstico de bifurcación.
No simétrica	LU general (SuperLU)	Backend universal. Cubre plasticidad no asociada, follower loads, contacto con fricción.

Origen de los flags, todos derivables del modelo:

- `is_symmetric`: AND lógico de `material.IS_SYMMETRIC` y `element.PRESERVES_SYMMETRY` sobre todos los componentes del dominio. Defaults `True`.
- `is_positive_definite`: arranca en `True` para `LinearSolver` y `NonlinearSolver`; en `False` para `ArcLengthSolver`. Se degrada automáticamente si Cholesky reporta no-positividad.
- `size`: número total de DOFs.

Ningún flag se pide al usuario.

EigenSolver (ADR 0009, problema generalizado). La capa algebraica incluye además `fenix.math.linalg.eigen.EigenSolver`, que resuelve el problema generalizado simétrico $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$ envolviendo `scipy.sparse.linalg.eigsh` (ARPACK Lanczos con shift-invert centrado en σ). No comparte el Protocol `LinearAlgebraSolver` de los backends de $K \cdot x = b$ porque su firma natural es `solve(K, M, n_modes) → (λ, φ)`. El `ModalSolver` lo invoca para el análisis modal; los cálculos internos de shift-invert generan una factorización de $(K - \sigma \cdot M)$ reutilizada en cada iteración Lanczos, así que la palanca de optimización es la misma — Cholesky enchufado en lugar de SuperLU bajaría 2× el coste del modal en problemas SPD (pendiente, ver memoria de cierre).

7.3 Fallback automático SPD → LU

Si en algún paso K deja de ser positiva definida (paso a régimen postcrítico, daño con reblandecimiento), Cholesky lanza `CholeskyNotPositiveDefiniteError`. Los tres solvers no lineales lo capturan y reinstancian el backend con LU general para el resto del análisis. La transición es silenciosa — solo se imprime un mensaje informativo en `stdout`.

Esto significa que **forzar Cholesky desde YAML como diagnóstico nunca rompe el análisis**: si la matriz se vuelve incompatible, el sistema se autocorrigie.

7.4 Factorización reusable y Newton modificado

La interfaz expone `solver.factorize(K)` que retorna un objeto `FactorizedSolver` con `solve(b)` reutilizable. Habilita:

- **Newton modificado:** el `NonlinearSolver` admite el parámetro `freeze_tangent_after_iter: int` que congela la factorización tras N iteraciones del paso. Útil cuando la tangente cambia poco y el coste dominante es la factorización.
- **Dinámica implícita lineal** (ADR 0009 fase 3): el `NewmarkSolver` factoriza una sola vez la matriz efectiva $A_{\text{eff}} = M + \gamma \Delta t \cdot C + \beta \Delta t^2 \cdot K$ al inicio del análisis y reutiliza la factorización en cada paso temporal. Con Δt constante y problema lineal, los cientos o miles de pasos del análisis se reducen a sustituciones triangulares baratas. Es el mismo `FactorizedSolver` del despachador.
- **Análisis futuros** (pandeo lineal, dinámica implícita no lineal con factorización congelada entre pasos cuando la tangente cambia poco) reutilizan la misma interfaz sin extensiones adicionales.

Ejemplo YAML:

```
solver:
  type: NonlinearSolver
  num_steps: 10
  tol: 1.0e-8
  freeze_tangent_after_iter: 2
```

7.5 Override desde YAML — herramienta de diagnóstico

Solo en caso de necesidad de *diagnóstico* o *benchmarking*, el bloque `solver` admite el campo opcional `linear_algebra`:

```
solver:
  type: LinearSolver
  linear_algebra: auto           # default; el despachador decide
  # linear_algebra: cholesky    # forzar Cholesky (requiere scikit-sparse)
  # linear_algebra: lu          # forzar LU (siempre disponible)
  # linear_algebra: ldlt        # placeholder; degrada a LU con warning
```

Cuándo usar el override:

- Comparar tiempos entre Cholesky y LU en un mismo problema.
- Forzar LU si se sospecha un bug en la auto-detección de simetría tras añadir un material nuevo.
- Reproducir un caso de regresión bit-a-bit con un backend específico.

Cuándo NO usarlo: como parte del setup habitual de un caso de análisis. No es decisión de modelado; el default `auto` cubre todos los regímenes correctamente.

7.6 Activación de Cholesky (opcional)

```
conda install -c conda-forge scikit-sparse
```

Una vez instalado, los problemas estáticos lineales y los Newton estables empiezan a usar Cholesky automáticamente — sin tocar YAML, sin recompilar, sin reescribir specs. Si la dependencia no está disponible, Fenix funciona idéntico con LU.